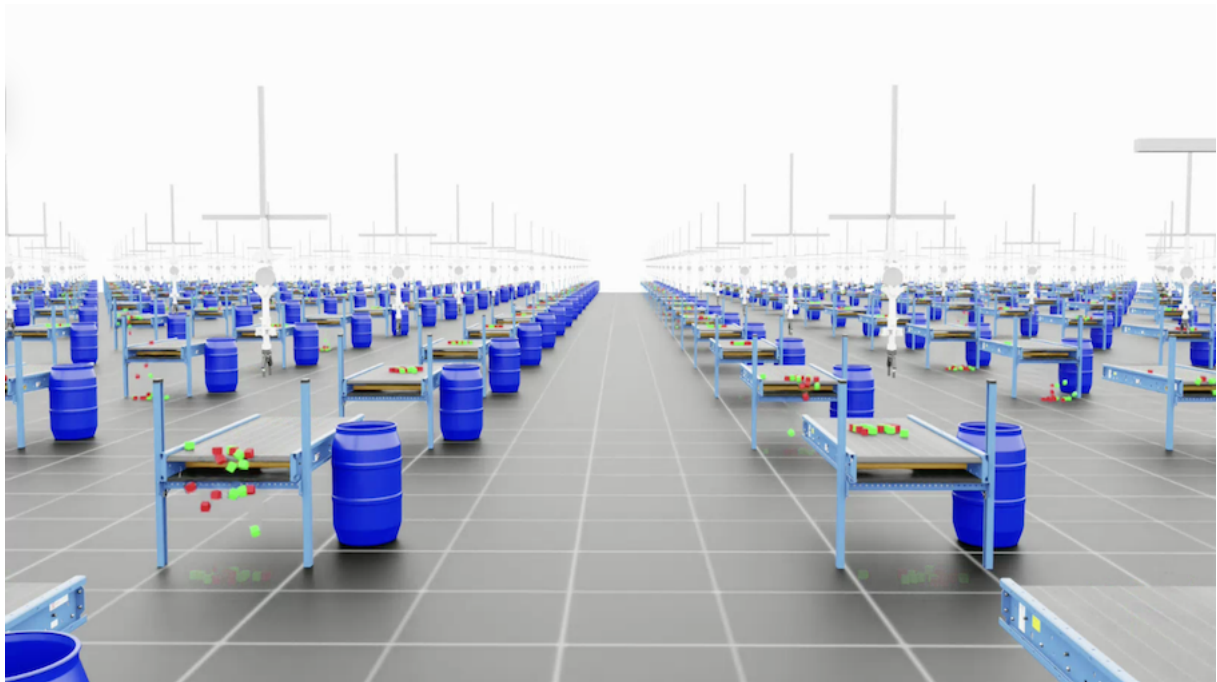


Simulative Construction and Verification of a Tendon-Driven Robot for Conveyor-Based Pick-and-Place Tasks Using Isaac Lab

Project Thesis im Studiengang Computational Engineering M.
Sc.

Friedrich-Alexander-Universität Erlangen-Nürnberg
Lehrstuhl für Fertigungsautomatisierung und Produktionssystematik
Prof. Dr.-Ing. J. Franke



Bearbeiter: Georg Meyer

Matrikelnr.: 22791103

Betreuer: Prof. Dr.-Ing. J. Franke
Dipl.-Ing. M. Landgraf

Abgabetermin: 01.07.2026

Bearbeitungszeit: x Monate

Erklärung

Ich versichere, dass ich die vorliegende Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen, den 13. März 2026

Georg Meyer

Contents

List of Figures	iv
List of Tables	v
List of Abbreviations	vi
List of Symbols.	vii
1 Introduction	1
1.1 Motivation and Context	1
1.2 Problem Statement and Objectives	2
1.3 Thesis Outline	3
2 Background	4
2.1 Reinforcement Learning for Robotic Manipulation	4
2.1.1 Learning objectives and algorithm families	5
2.1.2 Exploration and data collection in manipulation	6
2.1.3 Simulation-to-reality considerations	6
2.1.4 Proximal Policy Optimization (PPO)	7
2.1.5 Why PPO is often used for robotic manipulation	7
2.2 Physics Simulation in Isaac Sim 5.1	7
2.2.1 Rigid Body Dynamics.	9
2.3 NVIDIA Isaac Sim and IsaacLab Architecture	11
2.3.1 Isaac Sim in the Omniverse application stack.	12
2.3.2 Core simulation features in Isaac Sim 5.1	12
2.3.3 IsaacLab as a unified robot learning framework	14
2.3.4 Rationale for selecting Isaac Sim 5.1 with IsaacLab	16
2.3.5 Comparison to alternative simulation platforms.	16
2.4 Tendon-Driven Robot Kinematics and Modeling	18
2.4.1 Tendon-driven actuation principles	18
2.4.2 Kinematic modeling: from tendon space to task space	19
2.4.3 Simulation considerations for tendon-driven systems in Isaac Sim	20
2.4.4 Precision, dynamics, and compliance	21
2.4.5 Workspace analysis and dexterity metrics	21

2.5	Robot and Scene Description Formats	24
2.5.1	USD as a scene graph for simulation worlds	24
2.5.2	URDF as a robotics-oriented robot model format	25
2.5.3	Format use in this thesis and tendon-driven modeling workarounds	26
2.6	Prior Work	28
2.6.1	GPU-accelerated reinforcement learning for manipulation	29
2.6.2	Tendon- and cable-driven robot simulation	30
2.6.3	Workspace analysis methods	31
2.6.4	Identified gap	32
3	Research Gap	36
3.1	Critical Analysis of State of the Art	36
3.2	Research Gap and Justification	36
3.3	Refined Problem Definition	36
3.4	Success Criteria	36
4	Methodology	37
4.1	Overview of Approach	37
4.2	Simulation Environment Setup	37
4.3	Robot Model Integration	37
4.3.1	2-DoF Articulated Base (Linear Y-Z Stage)	37
4.3.2	3-DoF Tendon-Driven Arm	38
4.3.3	Robotiq 2F-140 Gripper Integration	39
4.3.4	Modular Assembly via Isaac Sim’s Robot Assembler	40
4.3.5	Control Gain Tuning and Validation	42
4.4	Reinforcement Learning Setup	43
4.5	Initial Experiments	43
4.5.1	Reach Pose Task	43
4.5.2	Simple Pick and Place Task	44
4.5.3		44
4.6	Advanced Task Implementation	44
4.7	Software and Hardware Integration	44
5	Results.	45
5.1	Preliminary Experiments	45
5.2	Performance of Tendon-Driven Robot on Simple Tasks	45
5.3	Cloth Manipulation Task Results	45

5.4 Summary of Key Results	45
6 Discussion	46
6.1 Evaluation of Outcomes	46
6.2 Comparison with Related Work	46
6.3 Limitations of the Study	46
6.4 Practical Implications	46
6.5 Recommendations and Lessons Learned	46
7 Conclusion	47
7.1 Conclusion	47
7.2 Outlook	47
Bibliography	48
Source Directory	58

List of Figures

2.1	RL agent–environment interaction.	5
2.2	Isaac Sim physics stack.	8
2.3	Isaac Sim and IsaacLab stack.	33
2.4	Tendon-to-task-space mapping.	34
2.5	Workspace voxel map.	34
2.6	Model and scene description pipeline.	35
4.1	Integrated tendon-driven robot model.	37
4.2	Exploded view of robot assembly.	41

List of Tables

2.1	Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX..	10
2.2	Qualitative comparison of simulators with respect to thesis requirements.. . .	17
2.3	Roles of Universal Scene Description (USD) and Unified Robot Description Format (URDF) in the simulation pipeline used in this thesis..	26
4.1	Actuated joints of the integrated robot and their motion ranges..	42

List of Abbreviations

API	Application Programming Interface
CAD	Computer-Aided Design
CPU	Central Processing Unit
DoF	Degree of Freedom
DR	Domain Randomization
dVRK	da Vinci Research Kit
FEM	Finite Element Method
FK	Forward Kinematics
GAE	Generalised Advantage Estimation
GPU	Graphics Processing Unit
IK	Inverse Kinematics
IL	Imitation Learning
MDP	Markov Decision Process
MJCF	MuJoCo XML Format
MPC	Model Predictive Control
MPM	Material Point Method
PBD	Position-Based Dynamics
PD	Proportional-Derivative
PGS	Projected Gauss–Seidel
PID	Proportional-Integral-Derivative
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Red-Green-Blue plus Depth
RL	Reinforcement Learning
ROS	Robot Operating System
SDF	Signed Distance Field
TGS	Temporal Gauss–Seidel
URDF	Unified Robot Description Format
USD	Universal Scene Description
VBD	Vertex Block Descent
XML	Extensible Markup Language
XPBD	Extended Position-Based Dynamics

List of Symbols

$\mathcal{A}$	Action space
A_t	Advantage estimator
ϵ	Clipping parameter (PPO)
γ	Discount factor
h	Simulation time step
$J(\theta)$	Expected discounted return
k_j	Stiffness parameter (PBD)
$\mathbf{M}$	Mass matrix
π_θ	Policy (parameterized by θ)
$r(s_t, a_t)$	Reward function
$\mathcal{S}$	State space
θ	Policy parameters

1 Introduction

1.1 Motivation and Context

Tendon-driven robots are articulated mechanisms in which joint motion is generated by tensile elements that transmit forces from remotely placed actuators to the joints. This design reduces distal mass and inertia, enables compact joint packaging, and introduces intrinsic mechanical compliance that can be beneficial for contact-rich manipulation tasks [1, 2, 3]. At the same time, the cable-based transmission introduces modeling complexity: tendon routing geometry, elasticity, friction, and hysteresis effects create a non-trivial mapping between actuator commands and joint motion that is difficult to capture with classical analytical models alone [4, 5].

Reinforcement Learning (RL) has emerged as a promising paradigm for learning robot control policies directly from interaction data, reducing the reliance on explicit analytical models and hand-crafted control strategies [6, 7]. In the context of robotic manipulation, RL has demonstrated the ability to discover effective policies for tasks involving continuous, high-dimensional control, contact-rich dynamics, and sparse feedback [7]. Modern physics simulators with Graphics Processing Unit (GPU)-accelerated parallel execution have become central enablers of this approach: by running thousands of environment replicas simultaneously, they provide the sample throughput required for stable policy learning while avoiding wear, risk, and slow data collection on physical hardware [8].

NVIDIA Isaac Sim and the associated IsaacLab framework provide an integrated toolchain for this simulation-driven development. Isaac Sim offers high-fidelity physics simulation based on PhysX 5 with articulated rigid-body dynamics, contact modeling, and GPU-accelerated pipelines within a Universal Scene Description (USD)-based scene representation [9]. IsaacLab builds on top of Isaac Sim and provides a unified, modular interface for designing robotics tasks as Markov Decision Processes (MDPs) and training policies at scale [10]. Together, they enable a workflow in which robot models are constructed, validated, and used for policy training within a single, reproducible environment.

This thesis is the first part of a two-part project conducted at the Institute for Factory Automation and Production Systems (FAPS) at Friedrich-Alexander-Universität Erlangen-Nürnberg. The broader project goal is to develop a multi-agent robotic system for automated textile sorting from mixed waste streams, where a tendon-driven robot picks

clothing items from a conveyor and a second robot arm stretches, inspects, and sorts them by condition. The present work establishes the simulation foundation and validates the tendon-driven robot model through basic manipulation tasks (reaching and placing rigid objects), providing the verified platform on which subsequent work will build.

1.2 Problem Statement and Objectives

Deploying a custom tendon-driven manipulator for conveyor-based pick-and-place tasks requires a faithful simulation model that accurately represents the robot’s kinematics, actuation characteristics, and workspace geometry. Without such a validated model, RL policies trained in simulation may learn to exploit simulator-specific artifacts rather than developing behaviors that transfer to the physical system [11, 12].

The specific robot considered in this thesis is a 5-Degree of Freedom (DoF) manipulator consisting of a 2-DoF prismatic positioning stage (providing linear motion in the y - and z -directions), a 3-DoF tendon-driven arm (one proximal bending joint and a 2-DoF cable-driven wrist), and a Robotiq 2F-140 parallel gripper as the end-effector. The design builds on the tendon-driven robot concept described by Klein [13]. Integrating this robot into a simulation-based RL pipeline requires addressing several interconnected challenges: constructing a modular simulation scene, faithfully representing the tendon-driven actuation mechanism, validating the simulated robot against established benchmarks, and formulating RL tasks that progressively build manipulation competence from basic skills to task-level behavior.

A further question addressed in this thesis is whether the compliant tendon-driven wrist mechanism, can be transferred to a standard industrial robot arm and whether such a transfer yields measurable benefits in manipulation performance. This experiment isolates the contribution of the wrist design from the overall robot morphology and provides insight into the generalizability of the tendon-driven approach.

The thesis pursues the following objectives:

1. **Validated tendon-driven robot model in simulation.** Construct and validate a modular simulation model of the 5-DoF tendon-driven manipulator in Isaac Sim that faithfully represents kinematics, actuation dynamics, and workspace geometry, such that it can serve as a reliable platform for RL-based policy training.
2. **Progressive RL-based manipulation skill acquisition.** Develop and evaluate RL policies for manipulation tasks of increasing complexity comparing Proportional-

Derivative (PD)-driven and tendon-driven actuation variants to quantify how actuation modeling influences learning performance.

3. **Transferability of the tendon-driven wrist mechanism.** Investigate whether the compliant 2-DoF tendon-driven wrist, improves manipulation capability when transferred to a standard industrial robot arm, thereby isolating the contribution of the wrist design from the overall robot morphology.
4. **Foundation for deformable-object manipulation.** Establish a documented, reproducible simulation infrastructure that forms the basis for subsequent work on deformable cloth handling, multi-agent textile sorting, and sim-to-real transfer in the master thesis.

1.3 Thesis Outline

This thesis is structured as follows. Chapter 2 introduces the fundamentals required for this work, covering RL for robotic manipulation with an emphasis on Proximal Policy Optimization (PPO) [14], rigid-body physics simulation in PhysX 5, the NVIDIA Isaac Sim and IsaacLab architecture [10], tendon-driven robot kinematics and modeling, and robot/scene description formats (USD and Unified Robot Description Format (URDF)). Chapter 3 identifies the specific research gap addressed by this thesis: the lack of a validated, open simulation model of a tendon-driven manipulator suitable for RL-based control in a conveyor-based pick-and-place setting.

Chapter 4 describes the methodological approach, including the simulation scene design, the modular robot model assembly, the tendon actuation implementation, the validation procedures (step-response analysis and workspace characterization), the MDP formulations for the reach and place tasks, and the RL training setup using `skrl` PPO [15, 14]. Chapter 5 presents the quantitative and qualitative results: robot validation metrics, learned policy performance for both actuation variants across both tasks, and the tendon wrist transfer experiment.

Chapter 6 discusses the implications and limitations of the approach, including simulation fidelity considerations, the effect of tendon-driven versus PD-driven actuation on policy learning, and the transferability of the wrist mechanism. Chapter 7 concludes the thesis and outlines the path toward the master thesis, where the validated simulation platform will be extended with deformable cloth simulation, multi-agent coordination, vision-based damage classification, and sim-to-real transfer preparation.

2 Background

2.1 Reinforcement Learning for Robotic Manipulation

Robotic manipulation requires generating control commands that achieve a goal while dealing with uncertainty, partial observability, contacts, and high-dimensional configuration spaces. Classical approaches rely on analytical models, trajectory optimisation, and carefully engineered feedback control. However, accurate models of frictional contact, deformable objects, or cluttered interactions are often difficult to obtain, and manually designing robust policies can be time-consuming. RL provides an alternative: it enables learning control policies from interaction data by optimising a task objective, thereby reducing reliance on explicit modelling and hand-crafted strategies. General RL background and formalism are treated in standard texts such as Sutton and Barto [6], and robotics-specific perspectives (including practical constraints and common failure modes) are discussed in Kober *et al.* [7].

MDP viewpoint and partial observability. A manipulation task can be modelled as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s_{t+1} \mid s_t, a_t)$ the transition dynamics, $r(s_t, a_t)$ the reward, and $\gamma \in [0, 1)$ the discount factor [6]. A policy $\pi_\theta(a \mid s)$ (parameterized by θ) induces trajectories $(s_0, a_0, r_0, \dots, s_T)$ and is trained to maximize the expected discounted return $J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$ [6]. In real manipulation systems, the full state s_t is typically not directly accessible; instead, control is based on observations o_t (e.g., proprioception, gripper state, pose estimates, perception features). This partial observability motivates careful observation design and, where appropriate, augmenting policies with memory (e.g., recurrent networks), or stacking observation histories to approximate the Markov property in practice [7].

Key challenges in manipulation. Three aspects commonly make RL for manipulation demanding: (i) **continuous, high-dimensional control**, where actions may correspond to joint torques, velocities, or Cartesian increments for many degrees of freedom; (ii) **contact-rich and nonlinear dynamics**, where small modelling errors in friction or contact impulses can lead to qualitatively different outcomes; and (iii) **sparse or delayed**

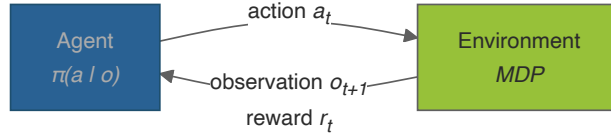


Figure 2.1: The reinforcement learning interaction loop: the agent selects action a_t from policy π , the environment transitions and returns observation o_{t+1} and reward r_t .

feedback, where success may only be measurable after long sequences of intermediate steps. Further practical constraints include safety, limited real-world interaction budgets, and the need for robustness across object variations and perception noise [7]. These characteristics influence algorithm choice, reward design, termination conditions, and the role of simulation during training.

Reward design and task specification. In RL, the reward function encodes the task objective. For manipulation, rewards are often composed of shaping terms (e.g., distance-to-target, alignment, grasp quality proxies) and sparse success signals (e.g., “object lifted above height”). Well-shaped rewards can accelerate learning but risk biasing the learned behavior toward unintended shortcuts; conversely, sparse rewards better reflect the true objective but can make exploration difficult. Robotics RL workflows therefore often iterate on reward definitions and termination criteria to balance learnability and task fidelity [7, 6].

2.1.1 Learning objectives and algorithm families

For continuous control, modern deep RL methods commonly use *policy gradient* and *actor-critic* approaches. Policy gradient methods directly optimise the policy parameters by ascending an estimate of $\nabla_{\theta} J(\theta)$, which naturally supports continuous actions through differentiable policy parameterizations [6]. Actor-critic methods introduce a learned critic (value function) that provides lower-variance learning signals for the actor, often improving stability and learning speed in practice [6]. A central quantity is the advantage function $A^{\pi}(s_t, a_t)$, which measures how much better an action is compared to the policy’s baseline behaviour at a state. In practice, advantage estimates $\hat{A}_t$ are computed from value predictions and collected rollouts, and then used to weight policy updates [6].

Bias-variance trade-off in advantage estimation. A practical challenge is balancing low-variance learning signals with accurate credit assignment over time. Generalised Advantage Estimation (GAE) provides a widely used mechanism to interpolate between

Monte Carlo returns and bootstrapped temporal-difference estimates by exponentially weighting temporal-difference residuals [16]. This improves learning stability in many continuous-control applications and is commonly paired with actor–critic training.

2.1.2 Exploration and data collection in manipulation

A defining challenge in RL is the *exploration–exploitation trade-off*: the agent must explore sufficiently to discover high-reward behaviours while exploiting learned knowledge to improve performance [6]. In manipulation, exploration is complicated by long horizons, sparse success signals, and irreversible or unsafe states (in real systems) [7]. Common practical measures include:

- **Stochastic policies and action sampling**, which induce exploratory behavior while still allowing gradient-based optimization [6].
- **Reward shaping and curricula**, where training starts from simplified conditions (e.g., easier initial object poses) and progressively increases difficulty to maintain a learnable signal [7].
- **Episode resets and termination design**, which affect state visitation and can substantially change the data distribution seen by the learner [7].

These aspects tie the learning algorithm closely to environment design: even a robust optimiser can fail if exploration is insufficient or if the reward/termination specification induces pathological behavior [7, 6].

2.1.3 Simulation-to-reality considerations

RL policies are frequently trained in simulation to avoid wear, risk, and slow data collection on real robots [7]. However, transferring policies to the real world is challenging due to the *reality gap*: discrepancies in geometry, contact parameters, actuator dynamics, sensor noise, and unmodeled effects can degrade performance. Common strategies include **domain randomization**, where visual and physical parameters are randomized during training so that the policy learns behaviors that are robust across a range of plausible conditions [17], and **dynamics randomization**, which targets uncertainties in physical parameters (e.g., mass, friction, motor gains, delays) to improve transfer for control policies [18].

2.1.4 Proximal Policy Optimization (PPO)

Among actor-critic methods, PPO is a popular choice due to its robustness and simplicity. PPO performs iterative policy improvement by collecting trajectories with the current policy and updating the policy with a surrogate objective that discourages overly large steps in policy space [14]. Define the importance ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. PPO maximizes the clipped objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \quad (2.1)$$

where $\hat{A}_t$ is an advantage estimate (commonly computed with GAE) [16, 14]. Intuitively, clipping limits the incentive to change the policy too aggressively, which helps prevent performance collapse from unstable updates. This is a practical consideration in contact-rich manipulation where learning can be sensitive to abrupt policy changes [14, 7].

2.1.5 Why PPO is often used for robotic manipulation

From a robotics perspective, PPO has several properties that make it attractive in manipulation pipelines:

- **Stable learning dynamics:** The clipped objective yields conservative and typically reliable updates [14].
- **Compatibility with continuous actions:** PPO naturally supports Gaussian policies over continuous controls [6].
- **Practical training loop:** The rollout-and-update structure integrates well with simulator-based training and supports systematic iteration on reward and termination design [7].

These considerations motivate PPO as a strong baseline for manipulation, especially when training stability and predictable iteration cycles are prioritized.

2.2 Physics Simulation in Isaac Sim 5.1

Physics-based simulation in Isaac Sim 5.1 is implemented as a layered stack that separates (i) scene and asset description, (ii) binding of these descriptions to a concrete backend, and (iii) numerical solvers that evolve the physical state over time. In the Omniverse ecosystem, physics properties are authored in USD through the `UsdPhysics` schema (engine-agnostic physics description) and `PhysXSchema` (backend-specific extensions), al-

lowing assets and environments to be defined declaratively in the scene graph [19, 20]. During simulation, the `omni.physx` runtime parses the USD stage, constructs the corresponding physics scene, advances the simulation, and writes back poses and state to USD [20]. Isaac Sim 5.1 uses NVIDIA PhysX 5 as its primary physics backend, providing rigid-body dynamics, articulations, contacts, and particle-based phenomena, with optional GPU-accelerated pipelines for throughput-critical workloads such as robot learning experiments with many parallel environments [21, 22, 23].

A detailed understanding of the underlying physics models and numerical methods is required for two reasons. First, sim-to-real transfer is fundamentally limited by discrepancies between simulated and real dynamics (the *reality gap*), where unmodelled effects, incorrect parameters (e.g., friction and damping), and solver artefacts can lead to policies that exploit simulator-specific regularities [24, 11]. Second, in RL, the simulator acts as a data-generating process; thus, modelling and solver choices directly shape the transition distribution used for learning, influencing both policy quality and generalization [11, 12]. In practice, this implies that solver selection, time-step choice, contact modelling, and collision geometry approximations must be treated as part of the learning system design, not as interchangeable implementation details [12, 25].

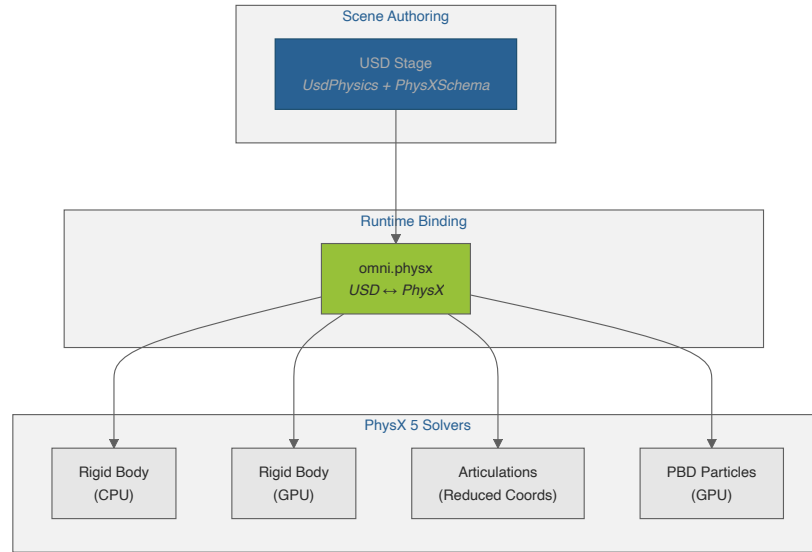


Figure 2.2: Conceptual physics simulation stack in Isaac Sim 5.1: physics is authored in USD via `UsdPhysics` and `PhysXSchema`, interpreted by `omni.physx`, and simulated using PhysX 5 with optional GPU-accelerated pipelines.

2.2.1 Rigid Body Dynamics

Rigid-body simulation assumes that each object moves as a rigid transform, i.e., deformation is neglected and the state is represented by position/orientation and linear/angular velocity. PhysX represents the simulated world as a set of actors (static, kinematic, dynamic) with attached collision shapes and material properties; dynamics are advanced under external loads (e.g., gravity), contact constraints, and joint constraints [22, 26]. In Isaac Sim, these concepts are exposed through USD physics schemas and their PhysX extensions, enabling rigid bodies, colliders, and constraints to be authored on scene primitives [21, 19].

Articulated bodies, joints, and robot mechanisms. Robotic manipulators are modelled as articulated mechanisms, i.e., rigid links connected by joints that constrain relative motion and optionally apply actuation (drives). PhysX provides an *articulation* abstraction that is simulated in reduced coordinates, where the configuration is parameterized by the root pose and joint coordinates rather than the world pose of each link [27]. This formulation is designed to improve numerical fidelity for mechanisms (e.g., reduced joint error and improved behavior under large mass ratios) compared to independently simulating rigid bodies connected by generic constraints [27]. In robotics applications, robot mechanisms such as tendon-driven arms and multi-finger grippers are simulated as chains of rigid links with joint constraints and joint drives, such that control inputs act on joint coordinates while the solver enforces inter-link constraints and contacts [27, 21]. PhysX also exposes explicit *articulation tendons* as additional constraint structures within an articulation, including fixed tendons (coupling joint coordinates) and spatial tendons (spring-damper and limit constraints defined over attachment trees across links), which are relevant when cable-like actuation or joint coupling must be represented at the physics level [27, 28].

Contacts and friction models. Rigid-body collision response is formulated as a constrained dynamics problem in which contacts prevent interpenetration and can transfer or dissipate energy through restitution and friction [29, 21]. In PhysX, friction and restitution are specified via materials attached to shapes; friction is parameterized by static and dynamic coefficients and can be combined for contacting pairs using configurable combine modes [26, 30]. Because contact-rich manipulation is common in robotic assembly, grasping, and textile handling, parameter choices for friction and restitution are directly coupled to learning outcomes and to sim-to-real robustness [11, 12].

TGS constraint solving in PhysX. Rigid-body dynamics with contacts and joints is solved in PhysX using iterative constraint solvers. PhysX supports a classic PGS-style solver and the newer TGS solver, with TGS introduced in PhysX 5.1 and recommended for improved convergence and joint drive accuracy in many scenarios [26]. Both solvers operate iteratively over constraint manifolds (contacts, joints, friction), and accuracy is traded for performance by limiting iteration counts. A key motivation for temporal formulations is the observation that, for stiff constraint systems, replacing multiple solver iterations at a long time step by multiple smaller substeps can yield superior stability and accuracy at comparable computational cost [31, 25]. This perspective is consistent with PhysX guidance to consider smaller time steps (or substepping) when joint stability and contact behavior are insufficient under a fixed number of iterations, rather than indefinitely increasing iteration counts [25].

Simulation parameters and numerical tradeoffs. For robotics and RL, the most consequential rigid-body parameters are those controlling temporal discretization, solver convergence, and contact generation. The simulation time step h determines how frequently dynamics are integrated and how often the policy interacts with the environment; overly large h can lead to missed collisions, unstable joint behavior, and biased contact impulses, whereas smaller h improves accuracy at increased compute cost [32, 25]. Solver iteration counts control how many passes are made over constraints; PhysX allows per-actor iteration counts (position and velocity iterations), and the effective iterations for a connected island are determined by the maximum requested within that island [32, 26]. Contact generation is influenced by the pair `contactOffset` and `restOffset`: contact points are generated once shapes are within the combined contact distance, while the rest offset defines the target separation distance at rest, enabling predictive enforcement and reducing jitter at the cost of introducing a small separation bias [33]. Finally, friction coefficients (static/dynamic) and restitution parameters are specified in materials and substantially affect sliding, sticking, and impact behavior [30, 26].

Table 2.1: Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX.

Parameter	Location	Primary effect
Time step h	Physics scene	Accuracy vs. throughput; contact/joint stability
Solver iterations	Actor / island	Constraint convergence; joint error; contact quality
<code>contactOffset</code> / <code>restOffset</code>	Shape	Contact prediction and resting separation
Friction coefficients	Material	Sliding/sticking behavior; grasp stability [30]

GPU-accelerated rigid bodies for parallel RL environments. PhysX provides a GPU rigid-body pipeline that accelerates broad-phase, narrow-phase, contact generation, and constraint solving on CUDA-capable hardware, while exposing the same API concepts as the Central Processing Unit (CPU) pipeline [23]. Large performance gains are expected when many rigid bodies are active, which aligns with robot learning workloads where hundreds to thousands of environment replicas are simulated in parallel to increase sample throughput [23, 8]. In IsaacLab, PhysX GPU simulation is enabled by default and can be configured explicitly; however, GPU simulation requires preallocation of internal buffers and may fail when insufficient capacities are specified, which makes sizing and profiling part of simulation configuration for large-scale training [34]. The broader motivation for GPU-resident simulation in RL is that physics stepping and policy learning can share device memory, avoiding CPU–GPU transfer bottlenecks; this design has been demonstrated to yield substantial throughput improvements in GPU-based robot learning platforms [8].

Collision detection and geometry approximations. Collision detection cost is highly sensitive to the chosen collider representation. Primitive shapes (e.g., spheres, boxes, capsules) provide the strongest performance and numerical robustness because intersection tests and contact generation are simple and well-conditioned [29]. Convex meshes provide a practical middle ground for complex objects by enabling efficient collision queries while approximating concave geometry; PhysX requires *convex mesh cooking* to transform input geometry into an optimized representation for collision detection [35]. In Isaac Sim, mesh colliders are often approximated by convex hulls by default for performance, with more expensive approximations available when needed [21]. When concavity is important for dynamic objects, convex decomposition (a set of convex parts) is typically preferred over a single triangle mesh collider because triangle-mesh contact generation is expensive and, depending on pipeline and version, may require additional structures. In PhysX 5.3+, dynamic triangle meshes in the GPU collision pipeline require an associated Signed Distance Field (SDF) to accelerate collision detection, increasing memory use but improving query performance [36]. Therefore, collider selection becomes a modelling choice that trades geometric fidelity against computational cost and solver stability, and this tradeoff must be balanced against the throughput requirements of RL training.

2.3 NVIDIA Isaac Sim and IsaacLab Architecture

NVIDIA Isaac Sim (version 5.1) is used as the primary simulation environment in this thesis because it integrates robotics-oriented scene authoring, high-fidelity multi-physics

simulation, and sensor-realistic rendering within a single Omniverse-based application stack [9]. In addition, the associated IsaacLab framework is used to structure the learning and evaluation workflow, because it provides a unified, modular interface for designing robotics tasks as MDPs and training policies at scale using GPU-accelerated parallel simulation [10]. The resulting toolchain aims to minimize integration overhead between simulation fidelity (e.g., contact-rich multi-body dynamics), robotic middleware interfaces, and the training pipeline used for robot learning.

2.3.1 Isaac Sim in the Omniverse application stack

Isaac Sim is implemented as a reference application on top of NVIDIA Omniverse and is designed to support development, simulation, and testing of AI-driven robots in physically based virtual environments [9]. A central architectural decision is the use of USD as the scene representation and interchange format: robotics assets and environments are expressed as a hierarchical scene graph with reusable primitives, enabling non-destructive composition and scalable instancing workflows [9]. This representation is used as the common “data layer” across physics, rendering, and robotics-specific tooling, which reduces interface mismatches between asset creation and simulation-time execution.

Isaac Sim is built on Omniverse Kit, which is a modular application framework based on lightweight plugins and an extension mechanism for composing functionality [9]. Consequently, Isaac Sim capabilities (e.g., physics interfaces, sensor simulation, and robotics middleware bridges) are exposed as extensions that can be enabled, configured, and scripted from Python, which supports both interactive development in the graphical application and headless execution in automated pipelines [9]. This extension-centric architecture is relevant to robot learning workloads because it enables the simulator to be configured as a reproducible experiment runtime, rather than as a monolithic interactive tool.

2.3.2 Core simulation features in Isaac Sim 5.1

Isaac Sim combines high-fidelity physics, photorealistic rendering, and robotics integration within a single runtime [9]. These features are not independent: physics fidelity affects contact-rich manipulation, rendering realism affects perception and sim-to-real transfer, and middleware integration affects how simulation is connected to external robot software stacks. The most relevant capabilities for this thesis are summarized below.

- **Omniverse and USD-centric scene representation:** Scenes are represented in USD, which serves as the unifying data interchange format for robot assets,

environments, and simulation metadata [9]. This enables structured scene graphs and supports scalable instancing patterns for large simulation workloads.

- **High-fidelity physics based on PhysX 5:** Isaac Sim uses a GPU-based PhysX engine and provides access to rigid-body and articulated dynamics required for robotics simulation [9]. GPU rigid-body simulation in PhysX provides GPU-accelerated broad phase, contact generation, and constraint solving when enabled, while other components remain on the CPU [23].
- **Photorealistic RTX-based sensor simulation:** Isaac Sim supports multi-sensor RTX rendering and uses GPU access to simulate sensors such as cameras and lidar in an industrially scalable manner [9]. RTX-based sensor simulation is exposed via dedicated extensions (e.g., RTX sensors) within the Kit extension framework [37].
- **Robotics middleware support via Robot Operating System (ROS) 2 bridging:** A dedicated ROS 2 Bridge extension is provided to publish and subscribe to ROS 2 topics and services from within Isaac Sim, enabling bidirectional communication between simulation and external ROS 2 systems [37].

2.3.2.1 GPU acceleration and parallel simulation

GPU acceleration in the Isaac ecosystem is achieved at multiple levels, ranging from physics backend execution to batched simulation of many environments. At the physics-backend level, PhysX GPU rigid bodies are configured at the scene level and enable GPU-accelerated rigid-body pipeline components including broad phase, contact generation, and the constraint solver [23]. Since not all features are executed on the GPU, the performance benefit depends strongly on scene composition and on whether large numbers of active bodies are simulated concurrently [23]. This architecture is aligned with robot learning workloads in which many similar environments are executed in parallel to increase sample throughput.

At the application level, Isaac Sim exposes explicit support for cloning and replicating environments. In particular, the `isaacsim.core.cloner` extension is provided to clone prims and environments efficiently and to optionally filter collisions across clones, which is a practical requirement when many environments share a single process runtime [37]. This cloning functionality forms a key bridge between a high-fidelity simulator and the throughput requirements of RL, since scalable policy training is typically bottlenecked by the rate of environment rollouts rather than by single-scene fidelity.

2.3.3 IsaacLab as a unified robot learning framework

IsaacLab is used in this thesis as the primary framework for structuring robot learning experiments on top of Isaac Sim. IsaacLab is described as the successor to Isaac Gym and is intended to extend GPU-native simulation into large-scale, multi-modal robot learning, while remaining integrated with Isaac Sim for high-fidelity physics and RTX-based sensing [10]. The framework is designed to unify workflows spanning RL, imitation learning, and motion planning, while exposing reusable abstractions for robot assets, environments, and training pipelines [10].

A key motivation for IsaacLab is the unification of two prior lines of development in the Isaac ecosystem. First, Isaac Gym demonstrated that physics simulation and policy learning can be executed entirely on the GPU with direct tensor access, yielding large speedups over CPU-based simulators when training complex robot policies [8]. Second, the Orbit framework established a modular environment library on top of Isaac Sim, providing benchmark tasks and supporting policy training and demonstration collection using GPU-based parallelization [38]. IsaacLab is positioned as a consolidated framework that carries forward the GPU-native paradigm of Isaac Gym and the modular task and environment design principles of Orbit, but within an Isaac Sim-integrated runtime that supports richer sensing and broader multi-physics capabilities [10].

2.3.3.1 Manager-based MDP workflow and code reuse

IsaacLab structures simulation tasks explicitly as MDPs, i.e., as environments that provide state (or observations), actions, and reward/termination signals suitable for learning. Within the manager-based workflow, a base environment is composed from reusable managers (e.g., action, observation, and event managers) and is then extended to an RL task environment by adding reward, termination, command, and curriculum components [39]. This decomposition is relevant for thesis-scale experimentation because it separates environment implementation from task specification: instead of modifying core environment classes directly, task-specific behavior is encoded in configuration objects, which enables consistent reuse across tasks and reduces duplication across experimental variants [39].

The manager-based formulation also encourages a stable interface between simulation and training. In the documented workflow, the `ManagerBasedRLEnv` interface returns MDP-relevant signals (including reward and termination status) while remaining compatible with standard environment creation patterns (e.g., via `gymnasium.make`), which supports integration with multiple RL libraries without rewriting the simulator interface layer [39]. The resulting abstraction is intended to enable systematic experimentation across robot

morphologies and task configurations while preserving a consistent training loop.

2.3.3.2 Distributed training support in IsaacLab

Robot learning in high-dimensional, contact-rich tasks often requires a large number of environment interactions, which motivates distributed execution. IsaacLab provides explicit support for multi-GPU and multi-node RL training, with documented workflows using PyTorch distributed tooling (e.g., `torchrun`) and a process-per-GPU execution model in which each process instantiates its own IsaacLab environment and synchronizes gradients across processes [40]. This design is aligned with on-policy RL settings, in which large batch sizes and parallel rollouts can substantially increase training throughput, provided that simulator stepping and observation extraction can be executed efficiently.

The documented multi-GPU pipeline is also operationally relevant: each process is responsible for its own environment instance and rollout collection buffers, while synchronization occurs only during gradient updates [40]. This reduces coupling between simulation instances and avoids centralized simulation bottlenecks, at the expense of higher per-process resource usage. For thesis experiments that require extensive policy iteration or hyperparameter exploration, this design provides a practical path toward scaling without abandoning the integrated Isaac Sim runtime.

2.3.3.3 Imitation learning and motion planning integration

Beyond RL, IsaacLab supports imitation learning workflows that combine human demonstration segments with automated planning. The SkillGen pipeline is documented as integrating motion planning into the IsaacLab Mimic demonstration workflow, thereby generating collision-free trajectories between skill segments using an external planner [41]. In this workflow, cuRobo is used as the underlying motion planner for generating smooth, collision-free motions, and the planner is explicitly described as GPU-accelerated [41].

cuRobo itself is formulated as a GPU-parallel motion generation approach for manipulators, with a design that exploits massive parallelism (e.g., many optimization seeds and parallel collision checks) to achieve low-latency planning and inverse kinematics computations [42]. The availability of such a planner within the IsaacLab ecosystem is relevant for tasks where expert trajectories must be generated efficiently, either to bootstrap learning (e.g., as demonstrations) or to define hybrid learning-and-planning pipelines.

2.3.4 Rationale for selecting Isaac Sim 5.1 with IsaacLab

The combined selection of Isaac Sim 5.1 and IsaacLab is driven by the need to simultaneously address (i) contact-rich manipulation in articulated robots and (ii) scalable learning infrastructure. These requirements are tightly coupled in the targeted thesis use case: reliable contact dynamics are essential for manipulation tasks, while learning-based policies require large numbers of rollouts to converge.

Scalability is addressed through GPU-native simulation and distributed training utilities. Isaac Gym established that GPU-resident simulation and learning with direct tensor access can yield large improvements in end-to-end training throughput compared to CPU-based simulation paired with GPU-based neural network training [8]. IsaacLab extends this paradigm within an Isaac Sim-integrated stack and provides documented support for multi-GPU and multi-node training, enabling scale-out execution for RL workloads when training is bottlenecked by rollout collection [10, 40]. When combined with Isaac Sim cloning utilities for replicated environments [37], the simulator can be configured to support large-scale parallel rollouts within a consistent runtime, which is particularly advantageous in on-policy scenarios where sample throughput dominates wall-clock training time.

Finally, robotics integration requirements are addressed at the middleware level. Isaac Sim exposes a ROS 2 Bridge extension for publishing and subscribing to ROS 2 topics and services through OmniGraph nodes and action graphs [37]. This bridge supports a common development pattern in which simulation is used for software-in-the-loop or hardware-in-the-loop testing, and it provides a practical interface to connect simulation-time controllers and perception to external robot software stacks without bespoke transport layers.

2.3.5 Comparison to alternative simulation platforms

The choice of simulator is constrained by trade-offs between fidelity, performance, and integration effort. For this thesis, the critical differentiator is the ability to combine contact-rich articulated manipulation with scalable learning infrastructure within a unified and scriptable runtime. Alternative platforms are briefly contrasted below to justify the selection of Isaac Sim 5.1 with IsaacLab for this specific use case.

MuJoCo is a widely used physics engine for model-based control that emphasizes efficient dynamics and contact simulation, and it is commonly adopted as a baseline simulator in continuous-control research [43]. Its design makes it well-suited for rigid-body control benchmarks and for iterative controller development when the primary bottleneck is per-step simulation performance. However, the thesis use case requires integrated sensor-

realistic rendering and a scalable parallel training framework within the same architecture, and these requirements typically necessitate additional tooling beyond a core rigid-body simulator.

PyBullet is widely used as a Python-accessible simulation interface built around the Bullet physics engine, and it is often adopted in robotics prototyping due to its ease of scripting and broad ecosystem integration [44]. The underlying Bullet engine is documented in the context of real-time physics simulation and is frequently used in interactive applications [45]. In practice, PyBullet is frequently selected when rapid iteration and lightweight deployment are prioritized, but the simulator stack used in this thesis additionally requires an integrated high-fidelity rendering and sensor simulation pipeline and a unified large-scale training framework, which are provided natively by Isaac Sim and IsaacLab rather than assembled from separate components.

SOFA is an open-source simulation framework primarily targeted at medical simulation and is designed around an advanced software architecture for building complex physical simulations by composing algorithms, solvers, and collision models [46]. This makes SOFA a strong candidate for high-fidelity soft-body modeling, including scenarios where accurate mechanics are required. Nevertheless, the overall thesis workflow requires scalable robot learning infrastructure with high-throughput parallel environment execution and integrated robotics middleware interfaces, which are not the primary design objectives of SOFA and typically require additional integration layers.

Table 2.2: Qualitative comparison of simulators with respect to thesis requirements.

Platform	Primary strength (per source)
Isaac Sim + IsaacLab	Integrated Omniverse stack with PhysX, RTX, and RL tooling [9, 10]
MuJoCo	Efficient physics engine for model-based control [43]
PyBullet/Bullet	Accessible real-time physics simulation ecosystem [44, 45]
SOFA	Modular framework for complex deformable simulation [46]

Overall, Isaac Sim 5.1 with IsaacLab is selected because it provides an integrated software architecture that aligns with the requirements of contact-rich manipulation and scalable robot learning. The Isaac stack combines a USD-based scene representation, PhysX-backed multi-physics simulation, RTX-based sensing, and robotics middleware integration in a single Omniverse runtime [9, 37]. IsaacLab further provides a principled MDP-based task design workflow and documented multi-GPU/multi-node training support, which reduces the engineering effort required to perform large-scale learning experiments while retaining simulation fidelity [39, 40, 10].

2.4 Tendon-Driven Robot Kinematics and Modeling

Tendon-driven robots are articulated mechanisms in which joint motion is generated by tensile elements (tendons) that transmit forces from remote actuators to the joints through pulleys, guides, or routing channels. In this actuation paradigm, the actuators and transmissions can be placed proximally, while the distal structure remains lightweight. This design is widely used in continuum and cable-driven mechanisms, particularly when low distal inertia, compact joint packaging, or intrinsic compliance is desired [1, 2]. At the same time, the mechanical transmission characteristics of tendons introduce additional modeling complexity. The mapping between actuator commands and joint motion depends on geometry (routing and moment arms), tendon elasticity, and non-idealities such as friction and hysteresis, which are frequently reported as dominant error sources in cable- and tendon-driven systems [4, 5].

2.4.1 Tendon-driven actuation principles

In tendon-driven architectures, actuation is typically realized by commanding tendon displacement (spooling length) or tendon tension. For common antagonistic arrangements, a joint is actuated by at least two tendons such that differential tension generates a net joint torque while co-contraction modifies apparent stiffness. When tendons span multiple joints, a single tendon displacement can induce coupled motions, which is advantageous for compact designs but complicates decoupled joint control [1]. The mechanical benefit of remote actuation is that motors and gear reductions can be located near the base, reducing distal mass and inertia and thereby enabling higher angular accelerations and dynamic performance for a given torque budget [2].

The same properties that reduce distal inertia also reduce transmission rigidity. Tendon stretch, cable-pulley compliance, and frictional effects can cause configuration-dependent motion errors, phase lags, and hysteresis loops under load. These effects are not only relevant for force control but also for kinematic accuracy because they break the assumption of a unique, memoryless mapping between actuator displacement and end-effector pose [4, 5]. As a result, tendon-driven systems are often controlled using either (i) local joint sensing (e.g., encoders placed near joints) to close the loop in joint space, or (ii) model-based compensation and identification to improve the actuator-to-joint mapping, depending on the available sensing and the targeted accuracy [4].

2.4.2 Kinematic modeling: from tendon space to task space

Rigid-link jointed mechanisms. For tendon-driven mechanisms built from rigid links and discrete joints, the kinematic structure in task space is formally identical to that of a direct-drive serial manipulator once the joint coordinates are defined. The principal difference is that the commanded variables are naturally expressed in tendon space (tendon lengths or tensions) rather than joint space. Consequently, the overall mapping is typically decomposed into two stages: (i) a transmission model from tendon variables to generalized joint coordinates, and (ii) a conventional rigid-body forward kinematic chain from joint coordinates to end-effector pose. In practice, the transmission model is required because routing geometry and tendon elasticity determine how a commanded spool displacement converts into a joint angle, and multi-joint tendons can create coupling terms that are absent in direct-drive designs [4].

Continuum and cable-driven bending mechanisms. For tendon-driven continuum robots, joint coordinates are not associated with discrete revolute joints but with distributed deformation of a flexible backbone. A widespread modeling approach is the piecewise constant-curvature assumption, in which the backbone is approximated as a sequence of constant-curvature arcs. This assumption reduces an effectively infinite-dimensional shape description to a finite set of arc parameters, enabling closed-form forward and differential kinematics for many designs [47]. The assumption is frequently used as a baseline model and remains attractive for control and planning because it yields tractable Jacobians and efficient computations. However, tendon friction and hysteresis can substantially degrade the predictive accuracy of purely geometric constant-curvature mappings, motivating extended models that incorporate frictional memory effects and tension propagation along the tendons [5].

Implications for inverse kinematics and control. In tendon-driven systems, the Inverse Kinematics (IK) problem is frequently formulated in actuator space (tendon lengths) rather than in joint space. Even if a rigid-link IK solution exists in terms of joint angles, the implementation of that solution on real tendon-driven hardware can be limited by transmission nonlinearities. In particular, hysteresis implies that the same tendon command can yield different end-effector poses depending on motion history and external loading, such that a purely kinematic IK solution may not be repeatable without feedback [5]. For learning-based control pipelines, this motivates the separation between an *idealized kinematic model* used for planning or workspace analysis and a *simulation/control model* that includes compliance and control bandwidth limitations to represent actuation uncer-

tainty [4].

2.4.3 Simulation considerations for tendon-driven systems in Isaac Sim

Tendon-driven mechanisms pose specific simulation challenges because their behavior depends on cable routing, compliance, and frictional effects that are not captured by joint kinematics alone. In USD-based simulation pipelines, these effects must either be represented explicitly using tendon/cable primitives or approximated through coupled joints and compliant actuation.

Articulation tendons and coupled DoFs. Omniverse Physics (PhysX backend) provides an articulation tendon concept with two main variants: fixed tendons (coupling joint coordinates) and spatial tendons (imposing constraints on distances between selected links through attachment points). These constructs are intended to represent coupled joint relationships and cable-like constraints inside articulated mechanisms [48]. In principle, joint coupling through fixed tendons can emulate certain tendon-driven topologies by enforcing linear relationships between generalized coordinates. Spatial tendons can represent a cable path between attachments but still abstract away detailed contact mechanics inside routing guides [48].

URDF mimic joints as a pragmatic approximation. When a robot is imported via URDF, coupled motion can be encoded by mimic joints. The Isaac Sim URDF importer can apply the PhysX mimic joint mechanism to joints carrying a mimic tag, enabling linear coupling between a reference joint and a mimicking joint [49]. This mechanism is frequently used to represent mechanically linked joints such as parallelogram linkages or differential couplings. For tendon-driven approximations, mimic joints can be used to enforce idealized coupling relationships that would otherwise arise from shared tendon routing. It is noted in the Omni Physics documentation that mimic joints can introduce numerical instability when enforced as stiff constraints and that compliance can be introduced to improve stability when competing constraints apply [48].

Limitations relevant to tendon-driven fidelity. Although tendon primitives and mimic couplings allow tendon-like behavior to be introduced into an articulation, several fidelity gaps remain for tendon-driven robots: (i) distributed friction along tendon routing and hysteresis are typically not captured by ideal constraints, (ii) tendon stretch

and tension propagation require tuned compliance parameters to remain stable at simulation time steps, and (iii) multi-contact routing through pulleys and guides is usually simplified to a small set of attachment points. These limitations are significant because tendon friction and hysteresis are reported to dominate motion errors in tendon-driven systems in applications such as surgical robotics, and improved prediction accuracy is obtained only when frictional effects are included in the mapping [5, 4]. As a consequence, tendon-driven robots are often simulated using a hybrid strategy: the *kinematic chain* is represented by standard joints, while tendon effects are represented by coupled joints (tendons or mimic constraints) and by compliant joint drives tuned to reproduce the apparent stiffness/damping observed in the physical system. Where higher fidelity is required, custom actuation models that map tendon commands to joint torques with friction/hysteresis compensation are required [4].

2.4.4 Precision, dynamics, and compliance

Tendon-driven designs differ from direct-drive manipulators in both achievable dynamics and precision characteristics. Reduced distal inertia can enable rapid wrist motions and high dynamic bandwidth in configurations that would be limited by motor mass and gear-box inertia if actuators were placed distally [2]. However, because the actuation forces are transmitted through compliant tendons and routed interfaces, the closed-loop behavior is sensitive to friction, elastic stretch, and hysteresis effects. These phenomena can manifest as configuration-dependent tracking error and reduced repeatability compared to stiff, direct-drive solutions, and they motivate calibration and model-based compensation strategies when high positional accuracy is required [4, 5].

The safety-performance tradeoff in physical interaction is classically addressed by introducing compliance in the actuation/transmission, which reduces impact severity and can improve robustness in contact-rich tasks at the cost of reduced rigidity [3]. Therefore, tendon-driven designs can be regarded as mechanically aligned with manipulation scenarios in which force limiting and safe contact are prioritized over sub-millimeter absolute accuracy.

2.4.5 Workspace analysis and dexterity metrics

The workspace of a serial manipulator is the set of all end-effector poses that are reachable under joint limits and kinematic constraints. Workspace analysis is used to verify that a robot can reach task-relevant regions and to identify configurations that should be avoided due to singularities or poor motion quality [50, 51].

Forward kinematics and sampling motivation. Forward kinematics (Forward Kinematics (FK)) is defined as the mapping from a joint configuration $\mathbf{q}$ to an end-effector pose, i.e., position and orientation of a designated tool frame. For a general serial chain, this mapping can be written as

$$\mathbf{T}_{\text{EE}} = f(\mathbf{q}), \quad (2.2)$$

where $\mathbf{T}_{\text{EE}} \in SE(3)$ denotes a homogeneous transformation of the end-effector frame. In many practical systems, analytical derivation of workspace boundary surfaces is intractable due to high DoF, joint coupling, or non-standard kinematics. In such cases, the workspace is frequently approximated numerically by sampling joint space and evaluating FK repeatedly. The Monte Carlo approach is widely used because it is simple to implement, directly incorporates joint limits, and does not require explicit boundary derivation [50].

Monte Carlo workspace sampling and voxel aggregation. In the Monte Carlo approach, configurations are sampled from a prescribed distribution over the admissible joint ranges. In the simplest implementation, each joint q_i is sampled independently and uniformly within its limit interval. For each sample $\mathbf{q}^{(k)}$, the corresponding end-effector pose is computed by FK. The resulting set of poses is then aggregated into a discrete spatial representation, e.g., Cartesian voxels, to estimate reachability statistics per region. This computation is conceptually aligned with established robotics software workflows that generate reachable workspace point sets paired with joint configurations [52].

When voxel aggregation is used, the workspace is partitioned into a regular grid of bins (voxels) in position space (and optionally orientation space). Each sample contributes to the occupancy count of the voxel containing the sampled pose. This discretization supports additional metrics beyond mere reachability, including density-based indicators of how many joint-space configurations map to a given region [51].

Yoshikawa manipulability. Dexterity is commonly evaluated using Jacobian-based indices. Let $J(\mathbf{q})$ denote the geometric Jacobian that maps joint velocities to end-effector twist. The Yoshikawa manipulability measure is defined as [53]

$$w(\mathbf{q}) = \sqrt{\det(J(\mathbf{q}) J(\mathbf{q})^\top)}. \quad (2.3)$$

This scalar is proportional to the volume of the manipulability ellipsoid and becomes zero at kinematic singularities where the Jacobian loses rank. Higher values indicate that

small joint motions can produce motion in a wider range of Cartesian directions, which is desirable for tasks requiring dexterous repositioning and local adjustments [53].

Inverse condition number as an isotropy index. A complementary measure is derived from the Jacobian condition number. Salisbury and Craig introduced the use of Jacobian conditioning to identify well-conditioned (isotropic) operating points and to quantify error amplification through the kinematic mapping [54]. Let $\sigma_{\min}$ and $\sigma_{\max}$ denote the minimum and maximum singular values of $J(\mathbf{q})$. The inverse condition number is defined as

$$\kappa_{\text{inv}}(\mathbf{q}) = \frac{\sigma_{\min}(J(\mathbf{q}))}{\sigma_{\max}(J(\mathbf{q}))}. \quad (2.4)$$

This index lies in $[0, 1]$, approaches zero near singular configurations, and equals one at perfectly isotropic configurations (equal singular values), where motion/force transmission properties are directionally uniform [54].

Reachability density. Reachability density is defined here as the number of sampled configurations whose end-effector poses fall within a given voxel. Under uniform joint sampling, this count is proportional to the joint-space volume that maps into the voxel region; thus, higher density indicates that more distinct configurations can realize poses in that region. This concept is closely related to workspace density formulations that assign each voxel the number of reachable points (or a normalized probability density) and interpret higher density as higher positional/orientational reach richness and, in discretely actuated systems, higher achievable pose resolution [51].

Implications for task-space design in learning-based manipulation. For manipulation tasks in cluttered scenes and for RL-based control, the workspace and dexterity metrics directly inform environment layout and action-space design. Task objects (e.g., pick targets, placement zones) are preferably positioned in regions with high reachability density and consistently high manipulability to reduce the probability that the robot is driven into near-singular configurations during exploration. Similarly, action bounds for end-effector motion primitives can be restricted to dexterous regions to improve learning stability and to reduce large corrective motions caused by kinematic ill-conditioning [53, 54, 51]. When tendon-driven compliance is present, these considerations become more important because poor kinematic conditioning can amplify transmission uncertainties into larger Cartesian errors, which in turn increases undesired contact forces during object interaction [4, 3].

2.5 Robot and Scene Description Formats

Robotic learning workflows in simulation typically require at least two layers of model description: (i) a *robot model* capturing kinematic structure and physical properties, and (ii) a *scene/world model* capturing the placement of robots and objects, lighting and sensors, and simulator-specific physics configuration. In the USD-based *Omniverse* ecosystem, the scene is represented as a composed USD stage, while robots are typically authored in robotics-oriented formats such as URDF and subsequently imported into USD for simulation. This section explains the role of USD and URDF in this separation of concerns and clarifies how both formats are used in this thesis.

2.5.1 USD as a scene graph for simulation worlds

Core scene description concept. USD is a system for encoding scalable, hierarchically organized data (static as well as time-sampled) with the primary purpose of interchange and augmentation across digital content creation applications [55]. Beyond being a file encoding, USD is designed to *compose* a single stage from scene description distributed across many layers and assets, supporting sparse overrides and asset aggregation via composition operators such as references and variants [55]. This composition-centric design is the main reason USD is used as the foundation of *NVIDIA Omniverse*-based applications, including robotics simulation: a simulation world can be assembled from reusable assets (robots, fixtures, conveyors, task objects), while experiment-specific changes (randomization parameters, object placements, material settings) can be authored as non-destructive overrides.

Extensibility via schemas. A central mechanism in USD is *schema-based extensibility*: standardized or custom schemas define what properties are available on a prim and how data are interpreted by downstream tools [55, 19]. For robotics simulation, this is crucial because a scene must not only store geometry and transforms, but also physics and sensor semantics. In practice, simulation-ready USD stages rely on multiple cooperating schema families: (i) rendering-related schemas (e.g., light prims), and (ii) physics-related schemas that define rigid bodies, collisions, articulation joints, and constraints, enabling a simulator backend to interpret the stage as a dynamical system.

Lighting and sensor-relevant elements. Lighting in USD is represented by the USD lighting schema *UsdLux*, which provides a portable representation for common light types

and attributes (e.g., intensity/exposure and color) intended for interchange across rendering environments [56]. This is directly relevant for robotics simulation, because perception pipelines and synthetic data generation depend on reproducible lighting setups. The use of common light schemas also reduces simulator lock-in at the scene-description level, since the same lighting structure can be understood by multiple Omniverse tools and render backends [56].

Physics properties encoded in USD. Physics semantics in USD are provided by *UsdPhysics*, which defines prim and property schemas for representing physics simulation state and configuration on a stage [19]. Within this schema family, rigid bodies, joints, and constraints are described as USD prims and applied Application Programming Interface (API) schemas, allowing physics engines to identify which parts of the stage are simulated and how they are constrained [19]. Joint actuation is commonly represented via drive attributes that behave as implicit damped springs, i.e., forces/accelerations are defined via stiffness and damping terms toward a target position/velocity [19]. In Omniverse, additional simulator-specific tuning parameters are exposed through *PhysX schema additions*; for example, PhysX joint schemas provide per-axis parameters and actuator performance characteristics extending the generic *UsdPhysics* joint representation [57].

2.5.2 URDF as a robotics-oriented robot model format

Scope and typical usage. URDF is an Extensible Markup Language (XML)-based format used in the *Robot Operating System* ecosystem for specifying robot geometry and organization [58]. A URDF model typically focuses on the robot as a kinematic structure, described by rigid links connected through joints, plus the geometric information used for visualization and collision checking, and the physical properties required for rigid-body simulation (e.g., link inertias) [59]. This focus makes URDF a common interchange format for robotic manipulators and mobile robots, especially when robots are also integrated into *ROS* pipelines for control, state publishing, and visualization [58].

Kinematic structure, joint limits, and inertial properties. Within the URDF workflow, links are augmented with collision geometry and inertial parameters, and joints are augmented with joint dynamics and limits. The ROS 2 URDF tutorials explicitly emphasize that collision and inertial properties are needed for simulation, and that joint dynamics parameters (e.g., friction) can be specified as part of the joint description [59]. Joint limits (position bounds, effort/velocity bounds) are also represented at the joint

level in URDF-based tooling, and are typically used both for controller safety and for ensuring physically plausible simulation states [59].

Structural limitations relevant to tendon mechanisms. A key limitation of URDF is that, in standard robotics tooling, only serial kinematic chains are supported in a direct and portable way; more complex kinematic dependencies are generally handled through additional simulator- or controller-specific mechanisms [60]. A commonly supported exception is the *mimic joint* concept, which allows joint coordinates to be coupled through a linear dependency. Mimic joints are explicitly described as a pragmatic abstraction used to represent simplified closed-loop dependencies or mechanically coupled degrees of freedom, such as belt-and-pulley systems [60]. This capability is relevant for tendon-driven or cable-driven mechanisms when coupling can be approximated as a simple linear relationship. However, tendon routing with wrapping, varying moment arms, elastic cable stretch, frictional loss, and multi-joint coupling is generally not fully representable with the standard URDF kinematic abstraction alone [60]. In such cases, either simulator-specific extensions or higher-level actuation models must be introduced.

2.5.3 Format use in this thesis and tendon-driven modeling workarounds

Responsibilities of each format in the pipeline. In the context of Isaac Sim and Isaac Lab, USD is treated as the *runtime scene format*: robots and environment assets are ultimately represented as USD so that they can be loaded, referenced, and simulated on a USD stage [61]. URDF is used as the *authoring and interchange format* for the robot model, capturing: (i) robot topology (links and joints), (ii) joint types and joint limits, and (iii) rigid-body physical properties (e.g., inertial parameters and collision geometry) required for simulation [59]. By contrast, USD is used to integrate the robot with the rest of the simulated world and to attach simulation-specific semantics (physics schema applications, light prims, and renderer-facing scene elements) [55, 19, 56].

Table 2.3: Roles of USD and URDF in the simulation pipeline used in this thesis.

Format	Primary scope	Typical role in simulation
USD	Scene graph with composition	World assembly, physics schemas, lighting
URDF	Robot kinematics + basic rigid-body data	Robot model interchange; converted into USD

Which format is used for the tendon-driven robot and why. In this thesis, the custom tendon-driven arm component is provided as a *modified* URDF description and is imported into Isaac Sim, where it becomes a USD robot asset for simulation. This choice is motivated by the practical availability of URDF as a widely used robotics interchange format and by the fact that Isaac Sim provides an explicit import workflow that converts URDF robot descriptions into USD assets for use on a USD stage [62]. In addition, Isaac Lab recommends converting external robot model formats into USD using the provided importers and then performing subsequent editing, tuning, and large-scale simulation using the USD representation [61]. The final integrated robot is therefore simulated as USD within Isaac Sim, consistent with Omniverse’s USD-native scene representation [61].

Integration of robot and scene as USD assets. Once the robot components exist as USD assets, integration into a complete simulation scene is performed by placing them on the USD stage together with the environment geometry and task objects (including manipulation targets and supporting structures) and then attaching physics and rendering semantics through the relevant schemas [19, 56]. In Omniverse-based robotics workflows, modular robot assemblies are commonly created by combining separate USD robot assets (e.g., a base, an arm, and a gripper) via fixed joints. Isaac Sim’s *Robot Assembler* tool is explicitly designed for this use case: it combines two USD assets through a physically simulated fixed joint and produces a resulting USD asset that can be saved and loaded as a single composite robot [63].

URDF modifications and tendon modeling workarounds. Standard URDF does not provide a portable, first-class representation of tendon routing with wrapping, elastic elements, and multi-joint cable coupling [60]. Tendon-driven mechanisms are therefore approximated in URDF-to-simulator pipelines by mapping tendon actuation to joint-level drive targets (position/velocity/effort) and by encoding only those couplings that can be captured with simple kinematic relationships (e.g., mimic joints) [60]. In Omniverse workflows, practical URDF-to-USD conversion constraints further motivate simplification: Isaac Lab explicitly documents that some URDF tags used in other simulators (e.g., `<transmission>` and `<gazebo>` tags) are not supported by the URDF importer, and that pre-processing may therefore remove such tags prior to conversion [61]. This limitation is particularly relevant for tendon-driven robots if tendon routing or actuation coupling has been encoded via transmission-like blocks: such elements may need to be removed or replaced by simulator-native constructs after import.

In the present thesis draft, the tendon routing/transmission constructs in the imported arm description are simplified such that each tendon-driven joint is represented as an

actuated revolute joint driven directly by joint targets rather than an explicit cable model. Moreover, joint limits are enforced in the description to improve stability, while complex tendon dynamics (stretch, multi-joint coupling, backlash) are not explicitly simulated at this stage.

Why PhysX tendon features are not used in the current implementation. Although PhysX articulations support tendon constraints at the articulation level, including fixed tendons (constraints on joint positions) and spatial tendons (distance constraints along attachment points) [64], limitations in simulation fidelity and behavior have been documented for articulation tendons in Omniverse Physics. Specifically, articulation tendons are listed among known limitations, and workarounds are recommended: mimic-joint style coupling is suggested for fixed tendon behavior, and external forces are suggested to mimic spatial tendon effects [65]. In this thesis, these limitations motivate a simplified approach in which tendon effects are approximated via direct joint drives (spring-damper-like actuation toward joint targets) rather than via explicit PhysX tendon constraints, thereby prioritizing stability and controllability of the learning environment over detailed tendon mechanics.

- **URDF role:** The tendon-driven arm geometry, kinematic chain, joint limits, collision shapes, and rigid-body inertial properties are authored in URDF and then converted for Omniverse simulation [59, 62].
- **USD role:** The robot (after conversion) and the environment assets are integrated on a composed USD stage; physics and lighting semantics are expressed via *UsdPhysics* and *UsdLux*, and PhysX-specific parameters are expressed via PhysX schema additions [19, 56, 57].
- **Tendon workarounds:** Due to representational limitations of URDF and documented limitations of articulation tendon fidelity in Omniverse Physics, tendon actuation is approximated through joint drives and coupling simplifications rather than explicit tendon constraints [60, 65].

2.6 Prior Work

This section reviews prior work along the three dimensions that converge in the present thesis: (i) Reinforcement Learning-based manipulation in GPU-accelerated simulators, (ii) simulation and control of tendon- and cable-driven robots, and (iii) workspace analysis methods for custom manipulators. A final subsection summarizes FAPS-internal predecessor work. The review focuses on contributions that directly inform the methodological

choices made in this thesis and concludes by identifying the specific gap that the present work addresses.

2.6.1 GPU-accelerated reinforcement learning for manipulation

Isaac Gym and large-scale parallel training. Makoviychuk *et al.* demonstrated that running both PhysX simulation and neural-network training entirely on GPU eliminates CPU–GPU data transfer bottlenecks and yields two to three orders of magnitude training speedup over CPU-based simulators [8]. In their Franka cube stacking benchmark, 16 384 parallel agents trained with PPO converged in under 25 minutes on a single A100 GPU, and a Shadow Hand cube rotation task achieved 20 consecutive successful rotations in approximately one hour [8]. These results established Isaac Gym as the foundational platform for GPU-accelerated RL and underpin the Isaac Sim/IsaacLab ecosystem used in the present work.

Contact-rich manipulation and assembly. Narang *et al.* introduced *Factory*, a suite of Isaac Gym environments for contact-rich assembly tasks that achieves a reported 20 000-fold speedup over prior state-of-the-art contact simulation through signed-distance-function collisions, contact reduction from approximately one million naive contacts to approximately 10 000, and a Gauss–Seidel solver [66]. Building on *Factory*, Tang *et al.* (*IndustReal*) transferred peg-insertion and gear-assembly policies from simulation to a real Franka Panda, reporting 83–99% success rates over 600 physical trials with realistic part clearances of $\leq 0.5\text{--}0.6\text{ mm}$ [67]. *IndustReal* introduced a simulation-aware policy update mechanism that detects unreliable simulator states and a sampling-based curriculum for progressive task difficulty—techniques that are directly adoptable for conveyor-based pick-and-place policy design [67].

Sim-to-real transfer for dexterous manipulation. Allshire *et al.* achieved an 83% real-world success rate for 6-DoF cube reposing on a TriFinger robot, trained entirely in simulation on a single GPU using PPO with asymmetric actor-critic and keypoint-based pose representations [68]. The keypoint representation was shown to provide more stable gradients than position-plus-quaternion encoding, and domain randomization over friction, mass, and sensor noise enabled zero-shot sim-to-real transfer with 16 384 parallel environments [68]. Handa *et al.* (*DeXtreme*) extended this line of work to agile in-hand manipulation with an Allegro Hand using automatic domain randomization, which dynamically adjusts randomization ranges without manual tuning [69]. Both works employ

PPO with asymmetric actor-critic, providing the privileged critic with full simulation state while limiting the actor to real-world-available observations [68, 69].

Frameworks: Orbit, Isaac Lab, and skrl. Mittal *et al.* presented *Orbit*, a modular design powered by NVIDIA Isaac Sim with GPU-based parallelization, wrapping four RL libraries (rl_games, RSL-RL, Stable-Baselines3, skrl) and providing 20+ benchmark tasks [38]. Orbit evolved into *Isaac Lab* in March 2024, achieving up to 1.6 million frames per second for state-based manipulation tasks and adding mimic joint support, closed-loop kinematic chains, and a deformable-object solver [10]. The **skrl** library provides native first-class support for Isaac Lab environments with competitive performance to rl_games on standard benchmarks while offering greater modularity through seven standalone components [15]. In the present thesis, Isaac Lab serves as the development framework and skrl provides the PPO training backend.

2.6.2 Tendon- and cable-driven robot simulation

Tendon-driven RL in MuJoCo. Shahid *et al.* modeled a bio-inspired “RoboStrich” arm using MuJoCo’s native tendon elements with attachment sites routed through pre-defined wire run-through holes, demonstrating that general actuators acting on tendons produce realistic tendon-contraction-to-joint-actuation coupling [70]. The study showed that curriculum-based RL outperforms vanilla SAC for reaching tasks on this underactuated tendon-driven manipulator by progressing from simple to complex targets [70]. Saito and Morimoto presented the first systematic RL comparison between a 9-tendon continuum arm and a 7-DoF rigid manipulator across multiple tasks (reaching, crank rotation, throwing, peg-in-hole), finding that continuum arms excel in uncertainty-rich and anisotropic tasks while rigid arms are superior for precision tasks [71]. Sim-to-real validation on a real pneumatic continuum robot confirmed the MuJoCo-based model’s validity [71].

Musculoskeletal simulation. Caggiano *et al.* (*MyoSuite*) provided an automated pipeline converting OpenSim musculoskeletal models to MuJoCo, validating forward kinematics, moment arms, and muscle forces across 39 muscle-tendon units in MyoHand (23 joints, 29 bones) [72]. MyoSuite achieves over two orders of magnitude speedup compared to OpenSim and offers 204 tasks spanning dexterous manipulation with antagonistic muscle-tendon actuation, including non-stationarity scenarios such as muscle fatigue and tendon-transfer surgery [72]. The OpenSim-to-MuJoCo validation methodology is relevant as a reference for validating the Isaac Sim tendon model developed in this thesis.

Cable-driven parallel robot simulation with RL. Dhakate *et al.* (*CaRoSaC*) used Unity3D with Obi Rope for flexible cable simulation via Extended Position-Based Dynamics, validated against a real industrial 4-cable-suspended parallel robot [73]. The TD3-based RL controller outperformed classical kinematic solvers in trajectory tracking, especially near workspace boundaries where force-distribution assumptions break down [73]. A two-stage training approach—first in a simplified no-sag environment, then in a full simulation with realistic cable sag—reduced training time while maintaining performance [73]. Garrido Campos *et al.* validated a cable-driven parallel robot simulation against a hardware prototype, achieving a peak position divergence of only 0.27% across various speed scenarios [74].

Tendon-driven modeling benchmarks. Lilge and Burgner-Kahrs comprehensively benchmarked four tendon-driven continuum robot modeling approaches—constant curvature, pseudo-rigid-body, and Cosserat rod formulations—under varying loads [75]. A key finding is that tendon force expressions differ across the literature and yield divergent results under external loading, establishing accuracy-versus-computation-time trade-offs that inform which level of fidelity a physics engine should reproduce [75].

2.6.3 Workspace analysis methods

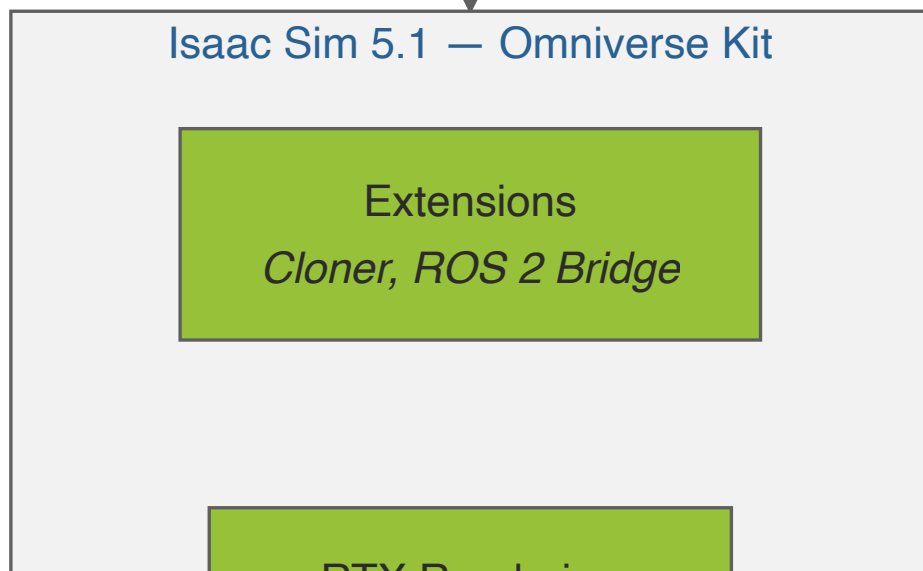
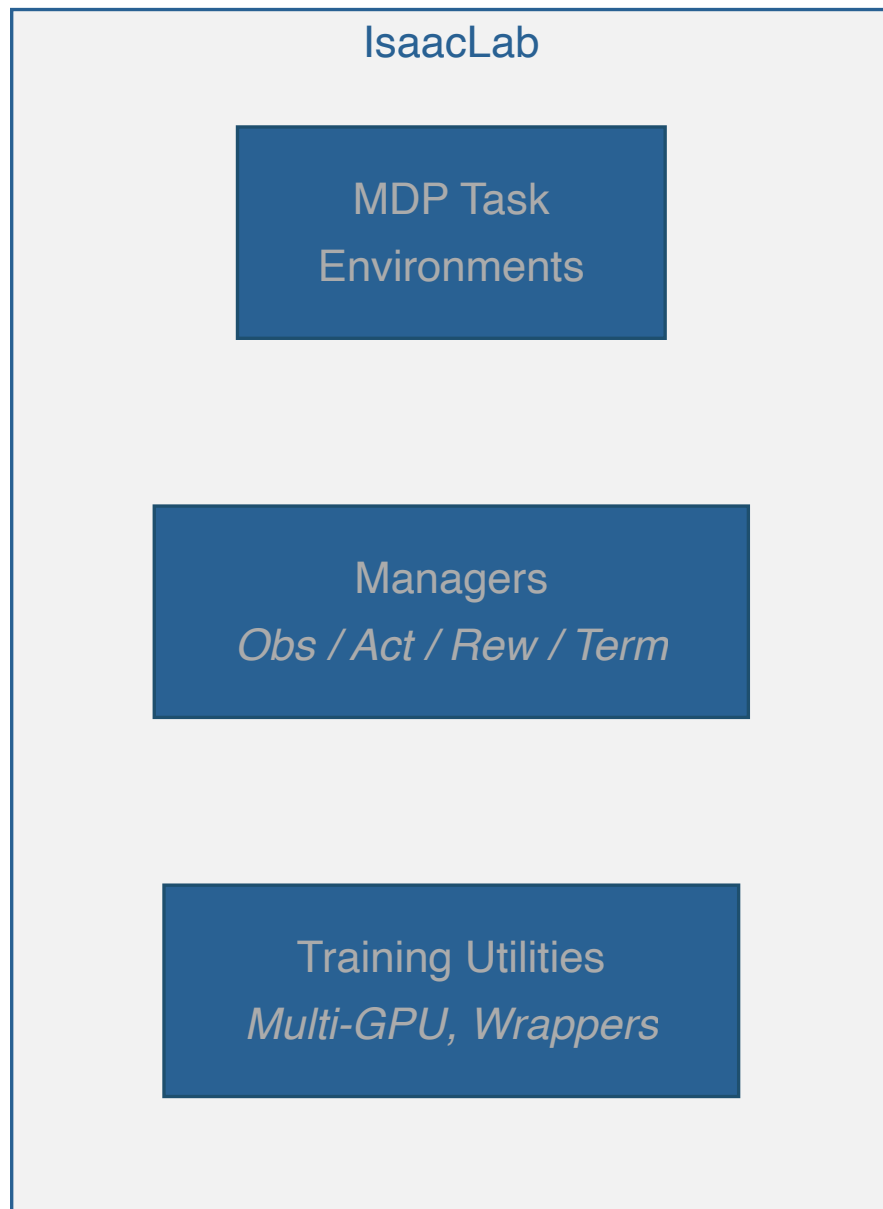
Manipulability measures. Yoshikawa defined manipulability as $w = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$, where $\mathbf{J}$ is the manipulator Jacobian, quantifying how well a manipulator can change end-effector position and orientation from a given configuration [53]. The resulting manipulability ellipsoid has the singular values of $\mathbf{J}$ as semi-axes, and $w = 0$ identifies singular configurations [53]. Vahrenkamp and Asfour extended this measure by penalizing Jacobian entries based on proximity to joint limits and self-collision distances, yielding a constrained manipulability metric that naturally reduces near workspace boundaries [76]. This constraint-penalized approach is directly applicable to tendon-driven manipulators, where cable tension limits act analogously to joint limits [76].

Computational workspace determination. Rastegar and Fardanesh established the Monte Carlo forward-kinematics sampling method for workspace determination, in which random joint-angle vectors are uniformly sampled within joint limits and mapped through forward kinematics to produce a workspace point cloud [50]. A known limitation is non-uniform density: uniform joint-space sampling produces denser coverage near the workspace center and sparser coverage at boundaries [50]. Peidr  *et al.* addressed this limitation with a Gaussian Growth method that iteratively densifies boundary regions,

achieving significantly more accurate workspace boundaries for high-DoF robots at comparable computation cost [77]. Cao *et al.* performed workspace analysis specifically on tendon-driven continuum robots, determining maximum bending angles from physical interference between tendon paths and the elastic backbone, and using a superposition method to combine individual module configuration spaces [78].

2.6.4 Identified gap

The literature reviewed above reveals a clear separation between two bodies of work. On one side, GPU-accelerated RL for manipulation has advanced rapidly within the Isaac Gym / Isaac Sim ecosystem, with demonstrated sim-to-real transfer for contact-rich tasks involving rigid-body robots [8, 66, 67, 68, 69]. On the other side, tendon- and cable-driven robot simulation for RL has been explored predominantly in MuJoCo [70, 72, 71, 43] or in specialized engines [73]. No published work, as of March 2026, combines tendon-driven robot actuation modeling with RL-based policy training in NVIDIA Isaac Sim or Isaac Lab. Furthermore, the FAPS-internal predecessor work by Klein [13] established workspace analysis and simulation for a cable-driven manipulator but did not employ learning-based control. The present thesis addresses this gap by constructing a validated tendon-driven robot model in Isaac Sim, formulating progressive RL tasks within Isaac Lab, and training manipulation policies using `skr1` PPO [15, 14].



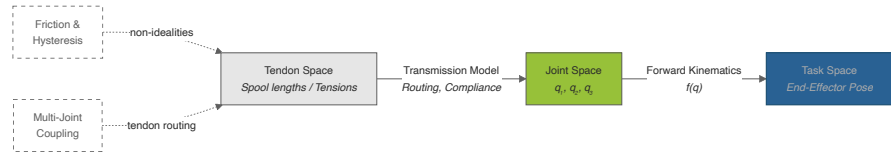


Figure 2.4: Two-stage kinematic mapping in tendon-driven mechanisms: tendon variables are mapped to joint coordinates through a transmission model that is affected by friction, hysteresis, and multi-joint coupling; joint coordinates are then mapped to end-effector pose through conventional forward kinematics.

Placeholder for *workspace voxelization and density visualization* image

Figure 2.5: Example visualization of a sampled reachable workspace. End-effector positions obtained from Monte Carlo FK sampling are aggregated into Cartesian voxels. Voxel occupancy represents reachability density, and additional overlays (e.g., manipulability) can be displayed to indicate dexterity variation across task space.

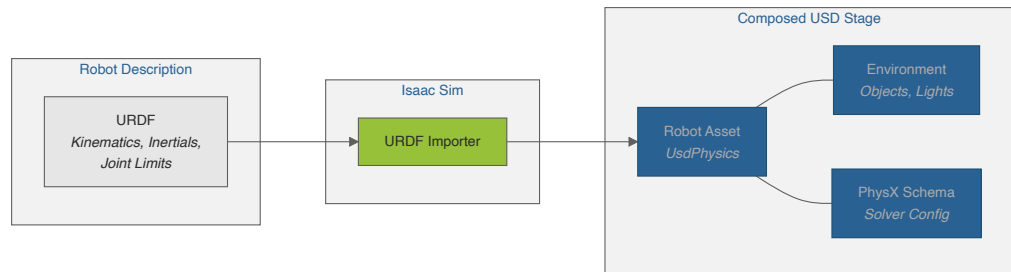


Figure 2.6: Schematic pipeline illustrating how robot descriptions (e.g., URDF) are converted into USD assets and integrated with environment elements (rigid objects, deformable bodies, lights, sensors) on a composed USD stage.

3 Research Gap

3.1 Critical Analysis of State of the Art

3.2 Research Gap and Justification

3.3 Refined Problem Definition

3.4 Success Criteria

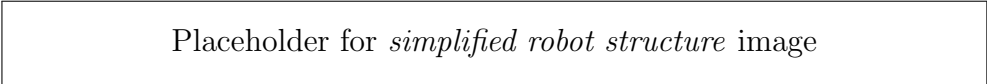
4 Methodology

4.1 Overview of Approach

4.2 Simulation Environment Setup

4.3 Robot Model Integration

In this section, the simulation setup for the custom tendon-driven robotic manipulator is presented. The complete robot model (Figure 4.1) consists of three modular components: a 2-DoF articulated base providing linear motion, a 3-DoF tendon-driven arm mounted atop the base, and a Robotiq 2F-140 two-finger gripper as the end-effector. All components are integrated in NVIDIA Isaac Sim 5.1.0, under the project’s Methodology and Implementation framework, to emulate the physical robot’s behavior in a virtual environment. The following subsections detail each component’s design and integration, the assembly of the full manipulator using Isaac Sim’s tools, and the validation of its control setup.



Placeholder for *simplified robot structure* image

Figure 4.1: Simplified structure of the integrated robot model in Isaac Sim, showing the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper attached as end-effector.

4.3.1 2-DoF Articulated Base (Linear Y–Z Stage)

The articulated base is a two-degree-of-freedom linear stage that provides planar positioning for the arm along the horizontal Y -axis and vertical Z -axis. It is designed exclusively in simulation, with no direct real-world counterpart, serving as a versatile foundation to adjust the arm’s position within the workspace. The base’s prismatic joints allow translation of the arm $\pm 0.4\text{m}$ laterally (Y direction) and -0.5m downward in height (Z direction) from a defined origin (with $Z = 0$ at the upper limit) - see Table 4.1 for a summary of joint travel ranges. These limits yield a horizontal span of 0.8m and a vertical adjustment of

0.5 m, chosen to cover the required operating area of the project’s frame while avoiding singular configurations or collisions at extremes.

In implementation, the base was constructed from scratch within Isaac Sim using the native USD robot schema definitions (rather than importing from URDF). It consists of rigid link geometry (modeled as simple shapes) and two linear actuators defined as prismatic joints along the global Y and Z axes. Each prismatic joint is configured with a physics drive (stiffness and damping) for position control, and appropriate limit stops at the specified travel range. The entire base assembly was built to fit the physical dimensions of the project’s frame and mounting interface, ensuring that when the arm is attached, its workspace aligns with the intended real-world environment. Because the base is purely virtual, it enabled rapid prototyping of different mounting heights and lateral positions for the arm without hardware modifications. All structural components of the base were kept simple and lightweight in simulation to maintain real-time performance.

4.3.2 3-DoF Tendon-Driven Arm

Mounted on the base’s vertical slider is the tendon-driven manipulator arm with three actuated degrees of freedom. This arm’s design and geometry are derived from a prior Computer-Aided Design (CAD) model developed in a master’s thesis by Klein (2023)[13]. The original arm is a cable-driven (tendon-driven) mechanism inspired by a human arm’s musculo-tendon structure, featuring an upper arm, forearm, and wrist. In our simulation, the arm was imported via a modified URDF file based on Klein’s CAD model. The URDF was adjusted for compatibility with IsaacSim - for example, custom transmission elements representing the tendon routing were simplified or removed, and each tendon-driven joint was defined as a standard revolute joint with direct actuation. The arm is rigidly attached to the top of the base (on the moving Z -axis platform) so that its shoulder reference frame coincides with the base’s moving frame.

Each of the arm’s three joints is modeled in IsaacSim as a revolute articulation with a position/force drive to emulate the tendon actuation. In the physical design, these joints would be driven by tendons via remote actuators, but in simulation they are controlled directly by setting joint angle targets and applying spring-damper forces. The first joint (shoulder/elbow pitch) allows approximately 172° of rotation ($\pm 86^\circ$, i.e. ± 1.5 rad, from neutral), corresponding to the primary bending of the “elbow” in the vertical plane as described by Klein [13]. The wrist mechanism of the arm is implemented as two consecutive revolute joints (perpendicular axes) representing the wrist’s flexion/extension and abduction/adduction; each of these has a range of $\pm 46^\circ$ (± 0.8 rad). These ranges are in line with the mechanical constraints reported in the original design [13]. For simulation

stability, hard joint limits are imposed in the URDF/USD definition to prevent the solver from pushing the joints beyond their intended travel.

Tendon Actuation Modeling: Instead of using PhysX’s built-in tendon simulation feature for articulations, the tendon-driven joints are simulated with direct joint drives. NVIDIA’s documentation notes that the PhysX articulation tendon feature can exhibit inadequate fidelity and unstable behavior under certain conditions [79]. In particular, spatial tendon constraints may lead to solver instabilities or unrealistic force feedback. To avoid these issues, we opted not to use PhysX “physxTendon” elements. Instead, each joint is given a PD control (a stiff spring with damping) that attempts to maintain the target angle, thereby mimicking the effect of a pretensioned tendon pulling the joint into position. The drive stiffness and damping values were initially chosen based on the estimated elasticity and friction in the tendons, and later fine-tuned using the Gain Tuning tool (see Section 4.3.5). This approach provides a stable yet simplified representation of tendon actuation: the joints behave as if driven by motors with compliance, without modeling cable stretch or complex tendon dynamics. We acknowledge that the tendon dynamics (e.g., stretch, backlash, and multi-joint coupling) are not explicitly simulated at this stage - incorporating a more realistic tendon model or a dedicated cable simulation is left for future work.

Collision Geometry: The arm’s collision shapes were generated using convex hull approximations of the CAD model geometry. Each link (upper arm, forearm, wrist links) is assigned a convex collision mesh computed from its visual mesh. Using convex hulls simplifies the collision geometry and is recommended for better physics performance in Isaac Sim. This ensures robust contact simulation while avoiding the computational cost and potential instability of complex mesh collisions. The convex colliders reasonably approximate the arm’s shape (covering joint protrusions, etc.) and were verified to avoid interpenetration in all valid joint configurations. With these collision models in place, the arm can interact with the environment (e.g., grasped objects or potential obstacles) in simulation without excessive physics solver load.

4.3.3 Robotiq 2F-140 Gripper Integration

For the end-effector, the Robotiq 2F-140 adaptive gripper was selected from IsaacSim’s built-in asset library. This is a two-finger parallel gripper well-suited to the project’s manipulation tasks. The 2F-140 model was chosen for its wide stroke and proven compatibility: it can grip a range of object sizes with an adjustable finger span up to 140mm, and it provides a significant grip force for secure holding. According to the manufacturer’s specifications, the 2F-140 offers a maximum stroke of 140mm and an adjustable grip force

from about 10N up to 125N[80]. It is rated for handling payloads up to roughly 2.5kg in a form-fit grasp, while the gripper’s own weight is about 1kg. These characteristics make it capable of picking and holding typical objects in our application (such as mixed waste items) without exceeding the arm’s load capacity.

In simulation, the Robotiq gripper is loaded as a self-contained articulated asset (with its finger linkage and drive mechanism pre-defined by NVIDIA’s asset). This greatly simplified integration: rather than modeling a custom gripper, the ready-made 2F-140 module was attached to the robot arm and controlled through its single actuated degree of freedom (the synchronized motion of the two fingers). The gripper’s coupling interface follows the standard ISO-50 flange, and it was directly compatible with the mounting point on the wrist of the tendon-driven arm. Notably, this gripper has been used in other Isaac Sim and Isaac Lab setups (e.g., with Universal Robots UR10 arms), indicating robust support and reliability in the simulation environment. Its inclusion here not only meets the functional requirements (adaptive gripping of various objects) but also leverages existing calibration and tuning knowledge from prior implementations. The official product sheet and instruction manual provided further technical details and usage guidelines, which were consulted to ensure the simulated gripper’s behavior (speed, force, fingertip friction, etc.) aligns with the real device [80].

4.3.4 Modular Assembly via Isaac Sim’s Robot Assembler

With the base, arm, and gripper defined as separate components, the complete robot was assembled within IsaacSim using the Robot Assembler tool. The Robot Assembler is an interactive utility provided by IsaacSim that allows users to combine multiple robot sub-assemblies into a single coherent model by creating fixed joints between them. In our case, we performed a two-step assembly: (1) attaching the 3-DoF tendon-driven arm onto the linear base, and (2) attaching the 2F-140 gripper onto the arm’s wrist flange. IsaacSim’s documentation explicitly supports these use cases - for example, assembling a robot arm to a moving base, or attaching a gripper to a robot arm, are typical workflows[79]. Using the Robot Assembler UI (accessible via Tools → Robotics → Asset Editor → Robot Assembler in Isaac Sim), the base was designated as the “Base Robot,” and the arm was added as the “Attach Robot” at the appropriate attach point on the base (the top plate of the Z-axis slider). The attach point corresponded to a pre-defined locator frame on the base where the arm’s bottom joint should coincide. Once the arm was assembled to the base, the combined base+arm was treated as a new single robot. Then, similarly, the gripper asset was attached to the arm: the arm’s URDF included a tool flange link at the wrist, which we used as the attach point for the gripper (with the gripper as “Attach Robot”

in the tool). The assembler automatically created a fixed joint linking the gripper’s base to the arm’s flange, aligning them based on their coordinate frames. After each assembly operation, the resulting composite robot (base+arm+gripper) was saved to a new USD file so that it can be loaded in the future as one complete manipulator model without needing to repeat the manual assembly steps.

Because the Robot Assembler uses a physically simulated fixed joint to bond the parts, the attached components behave as a single articulated body during simulation. This means the base, arm, and gripper move together without drift when physics is enabled, but they also remain modular in the project structure. One major advantage of this modular assembly approach is the flexibility for future hardware replacement or reconfiguration. Each subsystem (base, arm, gripper) can be swapped out or modified independently. For instance, if a different gripper model is to be tested in the future, it can replace the 2F-140 in the assembly with minimal effort; or if the project evolves to use a physical linear axis or a different arm, those could be integrated by the same method. The assembled manipulator’s design is therefore not monolithic - it is reconfigurable by design. Figure 4.2 illustrates an exploded view of the robot, showing the three modules (linear base, tendon arm, and gripper) and their attachment relationships. Such a view highlights the modular interfaces: the mounting plate between base and arm, and the wrist flange between arm and gripper.

Placeholder for *exploded view of assembly* image

Figure 4.2: Exploded view of the robot assembly, showing (from bottom) the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper. The arrows indicate the attachment points: the arm mounts onto the base’s top slide, and the gripper attaches to the arm’s wrist flange. This modular design allows each component to be independently developed or replaced.

After assembly, the final integrated robot has a total of five controlled joints (two prismatic joints in the base and three revolute joints in the arm, not counting the underactuated motion of the gripper’s fingers which are driven by one actuator). Table 4.1 summarizes the joint types and their ranges of motion in the integrated model. These ranges reflect both the mechanical design limits and the safety constraints imposed in simulation. By verifying the combined kinematic chain, we ensured that the robot can reach the required workspace for the intended tasks (e.g., covering the sorting area and adjusting to various bin heights) while respecting joint limits. All joints are configured with appropriate drive parameters and limit enforcement as discussed previously. In the next subsection, we address how the control gains for these joints were tuned and validated.

Table 4.1: Actuated joints of the integrated robot and their motion ranges.

Joint	Type & Axis	Range of Motion
Base Linear Y	Prismatic (horizontal slide)	−0.4 m to +0.4 m
Base Linear Z	Prismatic (vertical lift)	−0.5 m to 0.0 m (downwards)
Arm Joint 1	Revolute (shoulder/elbow pitch)	−86° to +86° (± 1.5 rad)
Arm Joint 2	Revolute (wrist tilt X-axis)	−46° to +46° (± 0.8 rad)
Arm Joint 3	Revolute (wrist tilt Y-axis)	−46° to +46° (± 0.8 rad)

4.3.5 Control Gain Tuning and Validation

Once the robot model was assembled, a critical step was to validate and tune the joint control gains to achieve stable and accurate motion. We utilized IsaacSim’s Gain Tuner extension for this purpose. The Gain Tuner provides tools to adjust the proportional position stiffness and damping parameters of each joint’s drive and to test the robot’s response to control inputs in simulation. According to NVIDIA’s documentation, the Gain Tuner is intended to help find a pair of stiffness and damping gains for each joint such that the robot can follow commanded trajectories with the expected behavior[79]. In practice, we performed automated gain testing on all five joints: the tool applied small-amplitude sinusoidal position commands and step changes within each joint’s allowable range, while we observed the joint’s motion response. For each joint, the stiffness (analogous to a P-gain) was increased until the joint responded quickly to commands without significant steady-state error, and the damping (D-gain) was adjusted to critically damp the response (minimizing overshoot and oscillation). Isaac Sim’s Gain Tuner provided visual feedback in the form of plots comparing commanded vs. actual joint trajectories, which was invaluable for iterative tuning and verifying stability.

Through this gain tuning process, we confirmed that the base’s linear axes and the arm’s revolute joints could be moved under position control with smooth and stable behavior. In particular, the heavy vertical axis (base Z) required a higher damping ratio to counteract gravity-induced oscillations, and the relatively lightweight wrist joints could use lower stiffness to reflect their tendon-driven compliance. After tuning, the robot’s joints exhibit near-critical damping: when given a step command, each joint reaches the target quickly (within 0.1-0.3 s settling time) with minimal overshoot (typically < 5% of the motion range) and no sustained oscillation. These tuned gains were then fixed in the simulation configuration for all subsequent experiments. This control validation step was essential to ensure that high-level control or planning algorithms (addressed in later chapters) would interact with a robot model that behaves in a realistic and predictable manner. By using

the Gain Tuner, we effectively calibrated the simulated actuators to mimic a well-tuned physical controller, thereby laying a sound foundation for the robot’s performance in simulation.

4.4 Reinforcement Learning Setup

In this thesis, PPO is used as the primary learning algorithm, together with GAE for advantage estimation, following standard actor–critic practice [16, 14]. The training loop is implemented with the `skrl` library, which provides modular PPO components (policy, value function, rollout storage, and optimization routines) and supports integration with simulator-based environments [15]. This allows the thesis to focus on environment formulation (observation design, action parameterization, reward shaping, termination conditions, and robustness measures such as randomization) while relying on a consistent and reproducible PPO implementation [15].

Reinforcement learning-based control is typically realized as an iterative interaction loop between an agent (policy) and an environment (simulation), in which behavior is improved by maximizing an expected return defined over episodes [6, 81]. In IsaacLab, task construction is formalized through an explicit MDP specification and a software architecture that decomposes environment logic into reusable terms for observations, actions, rewards, terminations, events, and curriculum scheduling [82, 83, 84]. This section describes the RL pipeline under this workflow and derives an MDP formulation suitable for the reach and place tasks in simulation.

4.5 Initial Experiments

To work towards the final implementation, initial experiments were conducted focusing on basic tasks such as reaching a specified pose and performing simple pick-and-place operations. These experiments served to validate the robot model and the reinforcement learning setup within the simulation environment. As comparison points, baseline results example implementations from IsaacLab with the UR10 robot arm were utilized.

4.5.1 Reach Pose Task

The reach pose task involved training the tendon-driven robot to move its end-effector to a designated position in 3D space. The performance was evaluated based on the accuracy of reaching the target position within a defined tolerance.

4.5.2 Simple Pick and Place Task

In the simple pick-and-place task, the robot was trained to grasp an object from one location and place it at another specified location. The success of this task was measured by the robot's ability to securely grasp the object and accurately place it at the target location.

4.5.3

4.6 Advanced Task Implementation

4.7 Software and Hardware Integration

5 Results

5.1 Preliminary Experiments

5.2 Performance of Tendon-Driven Robot on Simple Tasks

5.3 Cloth Manipulation Task Results

5.4 Summary of Key Results

6 Discussion

6.1 Evaluation of Outcomes

6.2 Comparison with Related Work

6.3 Limitations of the Study

6.4 Practical Implications

6.5 Recommendations and Lessons Learned

7 Conclusion

7.1 Conclusion

7.2 Outlook

Bibliography

- [1] RAO, Priyanka; PEYRON, Quentin; LILGE, Sven; BURGNER-KAHRS, Jessica: *How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance*, in *Frontiers in Robotics and AI*, 2021, Vol. 7, pp. 630245.
- [2] NEMOTO, Takeru; WALTER, Jonas; BACHMANN, Christian; GERLICH, Matthias; REITELSHÖFER, Sebastian; FRANKE, Jörg: *Highly Dynamic 2-DOF Cable-Driven Robotic Wrist Based on a Novel Topology*, in *IEEE Robotics and Automation Letters*, 2022, Vol. 7 (2), pp. 5727–5734.
- [3] BICCHI, Antonio; TONIETTI, Giovanni: *Fast and “Soft-Arm” Tactics: Dealing with the Safety-Performance Tradeoff in Robot Arms Design and Control*, in *IEEE Robotics & Automation Magazine*, 2004, Vol. 11 (2), pp. 22–33.
- [4] MIYASAKA, Muneaki; HAGHIGHIPANAH, Mohammad; LI, Yangming; MATHESON, Joseph; LEWIS, Andrew; HANNAFORD, Blake: *Modeling Cable-Driven Robot With Hysteresis and Cable-Pulley Network Friction*, in *IEEE/ASME Transactions on Mechatronics*, 2020, Vol. 25 (2), pp. 1095–1104.
- [5] KATO, Takahisa; OKUMURA, Ichiro; KOSE, Hidekazu; TAKAGI, Kiyoshi; HATA, Nobuhiko: *Tendon-driven continuum robot for neuroendoscopy: validation of extended kinematic mapping for hysteresis operation*, in *International Journal of Computer Assisted Radiology and Surgery*, 2016, Vol. 11 (4), pp. 589–602.
- [6] SUTTON, Richard S.; BARTO, Andrew G.: *Reinforcement Learning: An Introduction*, second edition, The MIT Press, 2018, ISBN 9780262039246.
- [7] KOBER, Jens; BAGNELL, J. Andrew; PETERS, Jan: *Reinforcement Learning in Robotics: A Survey*, in *The International Journal of Robotics Research*, 2013, Vol. 32 (11), pp. 1238–1274.
- [8] MAKOVYCHUK, Viktor; WAWRZYNIAK, Lukasz; GUO, Yunrong; LU, Michelle; STOREY, Kyle; MACKLIN, Miles; HOELLER, David; RUDIN, Nikita; ALLSHIRE, Arthur; HANDA, Ankur; STATE, Gavriel; GIRDHAR, Ankur; KUMAR, Tushar; FINK, Jonathan; MEREL, Josh; TASSA, Yuval: *Isaac Gym: High Performance GPU Based Physics Simulation for Robot Learning*, in *NeurIPS 2021 Datasets and Benchmarks Track (Round 2)*.

-
- [9] NVIDIA: *What Is Isaac Sim?*, 2026, isaac Sim Documentation (v5.1.0), last updated Jan 28, 2026,
URL <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html> (Accessed on 2026-03-03).
- [10] NVIDIA; MITTAL, Mayank; ROTH, Pascal; TIGUE, James; RICHARD, Antoine; ZHANG, Octi; DU, Peter; SERRANO-MUÑOZ, Antonio; YAO, Xinjie; ZURBRÜGG, René; RUDIN, Nikita; WAWRZYNIAK, Lukasz; ET AL.: *Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning*, arXiv, 2025, doi:10.48550/arXiv.2511.04831, arXiv:2511.04831,
URL <https://arxiv.org/abs/2511.04831> .
- [11] SALVATO, Erica; FENU, Gianfranco; MEDVET, Eric; PELLEGRINO, Felice Andrea: *Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning*, in IEEE Access, 2021, Vol. 9, pp. 153171–153187.
- [12] PENG, Xue Bin; ANDRYCHOWICZ, Marcin; ZAREMBA, Wojciech; ABBEEL, Pieter: *Sim-to-Real Transfer of Robotic Control with Dynamics Randomization*, in 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 3803–3810.
- [13] KLEIN, Max: *Arbeitsraumanalyse, Simulation und Bewegungsplanung eines seilgetriebenen robotischen Manipulators*, Masterarbeit, Friedrich-Alexander-Universität Erlangen-Nürnberg, 2023.
- [14] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec; KLIMOV, Oleg: *Proximal Policy Optimization Algorithms*, in arXiv preprint arXiv:1707.06347, 2017.
- [15] SERRANO-MUÑOZ, Antonio; CHRYSOSTOMOU, Dimitrios; BØGH, Simon; ARANA-AREXOLALEIBA, Nestor: *skrl: Modular and Configurable Library for Reinforcement Learning*, in Journal of Machine Learning Research, 2023, Vol. 24 (254), pp. 1–7.
- [16] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael I.; ABBEEL, Pieter: *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, in arXiv preprint arXiv:1506.02438, 2015.
- [17] TOBIN, Josh; FONG, Rachel; RAY, Alex; SCHNEIDER, Jonas; ZAREMBA, Wojciech; ABBEEL, Pieter: *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, in Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).

-
- [18] PENG, Xue Bin; ANDRYCHOWICZ, Marcin; ZAREMBA, Wojciech; ABBEEL, Pieter: *Sim-to-Real Transfer of Robotic Control with Dynamics Randomization*, in arXiv preprint arXiv:1710.06537, 2018.
- [19] PIXAR ANIMATION STUDIOS: *UsdPhysics: USD Physics Schema*, 2021, openUSD API documentation; generated on 2021-07-15.,
URL https://openusd.org/docs/api/usd_physics_page_front.html (Accessed on 2026-03-03).
- [20] NVIDIA: *Omni PhysX — Overview*, 2026,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/extensions/runtime/source/omni.physx/docs/index.html (Accessed on 2026-03-03).
- [21] NVIDIA: *Physics Simulation Fundamentals — Isaac Sim 5.1.0 Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/physics/simulation_fundamentals.html (Accessed on 2026-03-03).
- [22] NVIDIA: *Welcome to PhysX — PhysX 5.1.3 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/index.html> (Accessed on 2026-03-03).
- [23] NVIDIA: *GPU Rigid Bodies*, 2022, physX 5.1.0 documentation, last updated Nov 08, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/GPURigidBody.html> (Accessed on 2026-03-03).
- [24] JAKOBI, Nick; HUSBANDS, Phil; HARVEY, Inman: *Noise and the Reality Gap: The Use of Simulation in Evolutionary Robotics*, in *Advances in Artificial Life (ECAL 1995)*, *Lecture Notes in Computer Science*, Vol. 929, Vol. 929, pp. 704–720, Springer.
- [25] NVIDIA: *Best Practices Guide — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/BestPractices.html> (Accessed on 2026-03-03).
- [26] NVIDIA: *Rigid Body Dynamics — PhysX 5.1.1 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.1/docs/RigidBodyDynamics.html> (Accessed on 2026-03-03).

-
- [27] NVIDIA: *Articulations — PhysX 5.1.3 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/docs/Articulations.html> (Accessed on 2026-03-03).
- [28] NVIDIA: *Tendon Visual Authoring — Omni Physics Documentation*, 2025,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/extensions/ux/source/omni.usdphysics.ui/docs/dev_guide/tendon_authoring.html (Accessed on 2026-03-03).
- [29] NVIDIA: *Rigid Body Collision — PhysX 5.1.0 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/RigidBodyCollision.html> (Accessed on 2026-03-03).
- [30] NVIDIA: *PxMaterial — PhysX 5.1.0 API Reference*, 2022,
URL https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/_build/physx/latest/class_px_material.html (Accessed on 2026-03-03).
- [31] MACKLIN, Miles; STOREY, Kier; LU, Michelle; TERDIMAN, Pierre; CHENTANEZ, Nuttapong; JESCHKE, Stefan; MÜLLER, Matthias: *Small Steps in Physics Simulation*, 2019,
URL <https://mmacklin.com/smallsteps.pdf> (Accessed on 2026-03-03).
- [32] NVIDIA: *Simulation — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Simulation.html> (Accessed on 2026-03-03).
- [33] NVIDIA: *Advanced Collision Detection — PhysX 5.1.2 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.2/docs/AdvancedCollisionDetection.html> (Accessed on 2026-03-03).
- [34] NVIDIA: *isaacsim — IsaacLab Documentation (Simulation Configuration and GPU Physics)*, 2025,
URL <https://isaacsim.github.io/IsaacLab/main/source/api/lab/isaacsim.html> (Accessed on 2026-03-03).
- [35] NVIDIA: *Geometry — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Geometry.html> (Accessed on 2026-03-03).
- [36] NVIDIA: *Geometry — PhysX 5.3.0 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.3.0/docs/Geometry.html> (Accessed on 2026-03-03).

-
- [37] NVIDIA: *Isaac Sim: Extensions API*, 2025, isaac Sim Documentation (v5.1.0), last updated Oct 17, 2025,
URL <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/py/index.html>
(Accessed on 2026-03-03).
- [38] MITTAL, Mayank; YU, Felix; ZHAO, Jitendra; CHENG, Yuanhao; DIMITROV, Dimitar; QIU, Louis; VELONAKI, Mari; ZHU, Yuke: *Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments*, in IEEE Robotics and Automation Letters, 2023, Vol. 8 (6), pp. 3740–3747.
- [39] THE ISAAC LAB PROJECT DEVELOPERS: *Creating a Manager-Based RL Environment*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL https://isaac-sim.github.io/IsaacLab/v1.0.0/source/tutorials/03_envs/create_manager_rl_env.html (Accessed on 2026-03-03).
- [40] THE ISAAC LAB PROJECT DEVELOPERS: *Multi-GPU and Multi-Node Training*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/features/multi_gpu.html (Accessed on 2026-03-03).
- [41] THE ISAAC LAB PROJECT DEVELOPERS: *SkillGen for Automated Demonstration Generation*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/overview/imitation-learning/skillgen.html> (Accessed on 2026-03-03).
- [42] SUNDARALINGAM, Balakumar; HARI, Siva Kumar Sastry; FISHMAN, Adam; GARRETT, Caelan; VAN WYK, Karl; BLUKIS, Valts; MILLANE, Alexander; OLEYNIKOVA, Helen; HANDA, Ankur; RAMOS, Fabio; RATLIFF, Nathan; FOX, Dieter: *cuRobo: Parallelized Collision-Free Minimum-Jerk Robot Motion Generation*, arXiv, 2023, doi:10.48550/arXiv.2310.17274, arXiv:2310.17274,
URL <https://arxiv.org/abs/2310.17274> .
- [43] TODOROV, Emanuel; EREZ, Tom; TASSA, Yuval: *MuJoCo: A Physics Engine for Model-Based Control*, in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5026–5033.
- [44] PYBULLET CONTRIBUTORS: *Bullet Real-Time Physics Simulation: PyBullet*, 2026, project website,
URL <https://pybullet.org/> (Accessed on 2026-03-03).
- [45] COUMANS, Erwin: *Bullet Physics Simulation*, in *ACM SIGGRAPH 2015 Courses*.

-
- [46] ALLARD, J.; COTIN, S.; FAURE, F.; BENSOUSSAN, P.-J.; POYER, F.; DURIEZ, C.; DELINGETTE, H.; GRISONI, L.: *SOFA – an open source framework for medical simulation*, in *Studies in Health Technology and Informatics*, 2007, Vol. 125, pp. 13–18.
- [47] III, Robert J. Webster; JONES, Bryan A.: *Design and Kinematic Modeling of Constant Curvature Continuum Robots: A Review*, in *The International Journal of Robotics Research*, 2010, Vol. 29 (13), pp. 1661–1683.
- [48] NVIDIA: *Articulations – Omni Physics (Articulation Tendons and Mimic Joint Compliance)*, 2026,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/107.3/dev_guide/rigid_bodies_articulations/articulations.html (Accessed on 2026-03-03).
- [49] NVIDIA: *URDF Importer Extension – Isaac Sim Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/6.0.0/importer_exporter/ext_isaacsim_asset_importer_urdf.html (Accessed on 2026-03-03).
- [50] RASTEGAR, J.; FARDANESH, B.: *Manipulation workspace analysis using the Monte Carlo Method*, in *Mechanism and Machine Theory*, 1990, Vol. 25 (2), pp. 233–239.
- [51] DONG, Hui; DU, Zhijiang; CHIRIKJIAN, Gregory S.: *Workspace density and inverse kinematics for planar serial revolute manipulators*, in *Mechanism and Machine Theory*, 2013, Vol. 70, pp. 508–522.
- [52] THE MATHWORKS, INC.: *Generate reachable workspace of robot in environment (generateRobotWorkspace)*, 2026,
URL <https://www.mathworks.com/help/robotics/ref/generaterobotworkspace.html> (Accessed on 2026-03-03).
- [53] YOSHIKAWA, Tsuneo: *Manipulability of Robotic Mechanisms*, in *The International Journal of Robotics Research*, 1985, Vol. 4 (2), pp. 3–9.
- [54] SALISBURY, J. Kenneth; CRAIG, John J.: *Articulated Hands: Force Control and Kinematic Issues*, in *The International Journal of Robotics Research*, 1982, Vol. 1 (1), pp. 4–17.
- [55] PIXAR ANIMATION STUDIOS: *USD Frequently Asked Questions*, 2021, openUSD documentation (release 26.03); © 2021 Pixar Animation Studios.,
URL <https://openusd.org/release/usdfaq.html> (Accessed on 2026-03-03).

-
- [56] PIXAR ANIMATION STUDIOS: *UsdLux: USD Lighting Schema*, 2021, openUSD API documentation; generated on 2021-07-15.,
URL https://openusd.org/docs/api/usd_lux_page_front.html (Accessed on 2026-03-03).
- [57] NVIDIA: *Physx Joint Schema — Omni Physics Documentation*, 2026, omni Physics documentation; last updated on Jan 20, 2026.,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/joints/physx_joint_schema.html (Accessed on 2026-03-03).
- [58] OPEN ROBOTICS: *URDF (Unified Robot Description Format) — ROS 2 Documentation (Rolling)*, 2026, rOS 2 Rolling documentation (accessed 2026-03-03).,
URL <https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html> (Accessed on 2026-03-03).
- [59] OPEN ROBOTICS: *Adding Physical and Collision Properties to a URDF Model — ROS 2 Documentation (Foxy)*, 2026, rOS 2 Foxy (EOL) tutorial page (accessed 2026-03-03).,
URL <https://docs.ros.org/en/foxy/Tutorials/Intermediate/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.html> (Accessed on 2026-03-03).
- [60] THE HARDWARE_INTERFACE CONTRIBUTORS: *Joint Kinematics for ros2_control*, 2026, package documentation describing mimic joints and limitations of serial-chain URDF kinematics (accessed 2026-03-03).,
URL https://docs.ros.org/en/rolling/p/hardware_interface/doc/joints_userdoc.html (Accessed on 2026-03-03).
- [61] THE ISAAC LAB PROJECT DEVELOPERS: *Importing a New Asset*, 2026, isaac Lab documentation; last updated on Mar 03, 2026.,
URL https://isaac-sim.github.io/IsaacLab/main/source/how-to/import_new_asset.html (Accessed on 2026-03-03).
- [62] NVIDIA: *Tutorial: Import URDF — Isaac Sim Documentation (5.1.0)*, 2026, isaac Sim documentation; last updated on Jan 28, 2026.,
URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/import_urdf.html (Accessed on 2026-03-03).
- [63] NVIDIA: *Robot Assembler — Isaac Sim Documentation (5.1.0)*, 2026, isaac Sim documentation; last updated on Jan 28, 2026.,

-
- URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/robot_setup/assemble_robots.html (Accessed on 2026-03-03).
- [64] NVIDIA: *Articulations — PhysX SDK Documentation (5.5.0)*, 2024, physX documentation page including articulation tendons (fixed/spatial); dated Dec 04, 2024.,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.5.0/docs/Articulations.html> (Accessed on 2026-03-03).
- [65] NVIDIA: *Known Physics Limitations — Omni Physics Documentation*, 2026, omni Physics documentation; last updated on Jan 20, 2026.,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/guides/current_limitations.html (Accessed on 2026-03-03).
- [66] NARANG, Yashraj; STOREY, Kier; AKINOLA, Iretiayo; MACKLIN, Miles; REIST, Philipp; WAWRZYNIAK, Lukasz; GUO, Yunrong; MORAVANSZKY, Adam; STATE, Gavriel; LU, Michelle; HANDA, Ankur; FOX, Dieter: *Factory: Fast Contact for Robotic Assembly*, in *Robotics: Science and Systems (RSS)*.
- [67] TANG, Bingjie; LIN, Michael A.; AKINOLA, Iretiayo; HANDA, Ankur; SUKHATME, Gaurav S.; RAMOS, Fabio; FOX, Dieter; NARANG, Yashraj: *IndustReal: Transferring Contact-Rich Assembly Tasks from Simulation to Reality*, in *Robotics: Science and Systems (RSS)*.
- [68] ALLSHIRE, Arthur; MITTAL, Mayank; LODAYA, Varun; MAKOVICHUK, Viktor; MAKOVICHUK, Denys; WIDMAIER, Felix; WÜTHRICH, Manuel; BAUER, Stefan; HANDA, Ankur; GARG, Animesh: *Transferring Dexterous Manipulation from GPU Simulation to a Remote Real-World TriFinger*, in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11802–11809.
- [69] HANDA, Ankur; ALLSHIRE, Arthur; MAKOVICHUK, Viktor; PETRENKO, Aleksei; SINGH, Ritvik; LIU, Jingzhou; MAKOVICHUK, Denys; VAN WYK, Karl; ZHURKEVICH, Alexander; SUNDARALINGAM, Balakumar; NARANG, Yashraj; LAFLECHE, Jean-Francois; FOX, Dieter; STATE, Gavriel: *DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality*, in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5977–5984.
- [70] SHAHID, Asad Ali; TAKANISHI, Atsuo; KUO, Chung-Hsien: *Curriculum-Reinforcement Learning on Simulation Platform of Tendon-Driven High-Degree of Freedom Underactuated Manipulator*, in *Frontiers in Robotics and AI*, 2023, Vol. 10, pp. 1066518.

-
- [71] SAITO, Nozomi; MORIMOTO, Jun: *Characterization of Continuum Robot Arms Under Reinforcement Learning and Derived Improvements*, in *Frontiers in Robotics and AI*, 2022, Vol. 9, pp. 895388.
- [72] CAGGIANO, Vittorio; WANG, Huaijin; DURANDAU, Guillaume; SARTORI, Massimo; KUMAR, Vikash: *MyoSuite – A Contact-Rich Simulation Suite for Musculoskeletal Motor Control*, in *4th Conference on Learning for Dynamics and Control (L4DC)*, *PMLR*, Vol. 168, Vol. 168.
- [73] DHAKATE, Rohit; JANTOS, Thomas; ALLAK, Elvin; WEISS, Stephan; STEINBRENER, Jan: *CaRoSaC: A Reinforcement Learning-Based Kinematic Control of Cable-Driven Parallel Robots by Addressing Cable Sag Through Simulation*, in *IEEE Robotics and Automation Letters*, 2025.
- [74] GARRIDO CAMPOS, Julio; SILVA-MUÑIZ, Daniel; RIVERA-ANDRADE, Jorge; RIVEIRO, Eva: *Integrated Simulation and Control Environment for the Development and Safe Start-up of Cable Driven Parallel Robots*, in *International Journal of Computer Integrated Manufacturing*, 2024, Vol. 38 (7).
- [75] LILGE, Sven; BURGNER-KAHRs, Jessica: *How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance*, in *Frontiers in Robotics and AI*, 2021, Vol. 7, pp. 630245.
- [76] VAHRENKAMP, Nikolaus; ASFOUR, Tamim: *Representing the Robot’s Workspace Through Constrained Manipulability Analysis*, in *Autonomous Robots*, 2015, Vol. 38 (1), pp. 17–30.
- [77] PEIDRÓ, Adrián; REINOSO, Óscar; GIL, Arturo; MARÍN, José María; PAYÁ, Luis: *An Improved Monte Carlo Method Based on Gaussian Growth to Calculate the Workspace of Robots*, in *Engineering Applications of Artificial Intelligence*, 2017, Vol. 64, pp. 197–207.
- [78] CAO, Ke; KANG, Rongjie; BRANSON, David T.; GENG, Songjing; SONG, Zhibin; DAI, Jian S.: *Workspace Analysis of Tendon-Driven Continuum Robots Based on Mechanical Interference Identification*, in *ASME Journal of Mechanical Design*, 2017, Vol. 139 (6), pp. 062303.
- [79] NVIDIA: *Isaac Sim Documentation*, 2023, nVIDIA Omniverse Isaac Sim documentation,
URL <https://docs.omniverse.nvidia.com/isaacsim/latest/index.html> (Accessed on 2026-03-03).

-
- [80] ROBOTIQ INC.: *2F-85 and 2F-140 Adaptive Gripper Instruction Manual – Specifications*, 2019, product datasheet,
URL <https://robotiq.com/products/adaptive-grippers> .
- [81] PUTERMAN, Martin L.: *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, John Wiley & Sons, New York, NY, USA, 1994, ISBN 978-0-471-61977-2.
- [82] NVIDIA: *Manager Configuration — Getting Started With Isaac Lab*, 2026,
URL <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-second-robot-with-isaac-lab-03-implementing-mdp-with-managers.html> (Accessed on 2026-03-03).
- [83] THE ISAAC LAB PROJECT DEVELOPERS: *Task Design Workflows — Isaac Lab Documentation*, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/overview/core-concepts/task_workflows.html (Accessed on 2026-03-03).
- [84] THE ISAAC LAB PROJECT DEVELOPERS: *isaacsim.envs.mdp — Isaac Lab Documentation*, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaacsim.envs.mdp.html> (Accessed on 2026-03-03).

Source Directory